

# Fiat Shamir & Fischlin Transform

Tejas Pujari

December 16, 2025

# Identification Protocols

Motivation: Party A wishes to identify itself to party B to gain access to resources available at B.

# Identification Protocols

Motivation: Party A wishes to identify itself to party B to gain access to resources available at B.

Real World Scenarios:

- ▶ Opening a Door Lock
- ▶ Unlocking a Car
- ▶ Login at bank ATM.

Goal: Can we build ID protocols which realizes above scenarios **securely**.

# Attacks on ID protocols

Mainly, 3 types of attacks on identification schemes.

- ▶ Direct attacks = Try to break the Door lock.
- ▶ Eavesdropping attacks = Eavesdrop a network channel and steal information (stealing passwords directly)
- ▶ Active attacks = Install the fake ATM to steal the users credentials and later use this on his behalf with the real ATM.

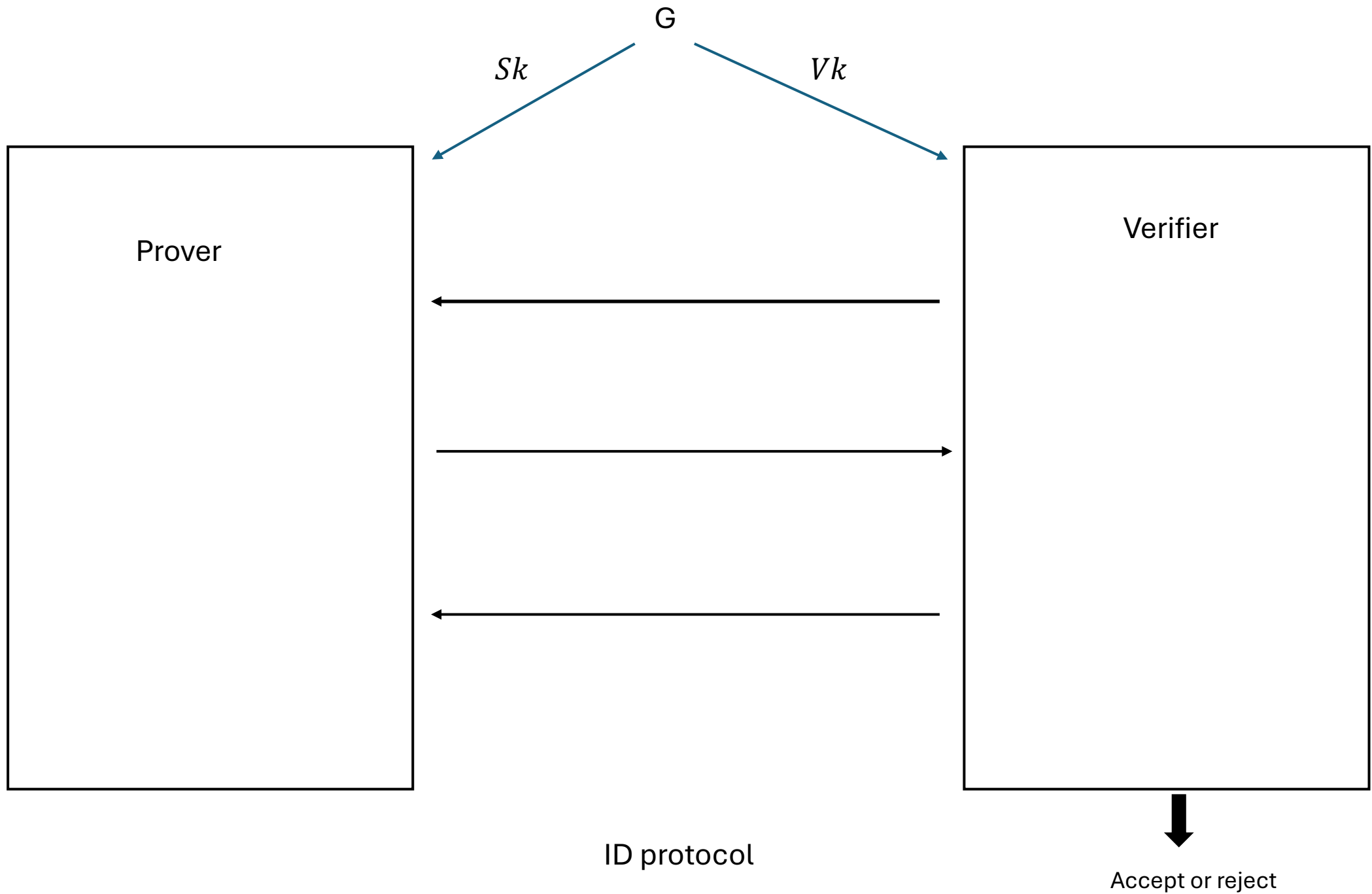
Adversary Goal: The underlying objective of all the attacks above is **impersonation**.

## Formal Definition

**Identification protocol:** An identification protocol is a triple  $I = (G, P, V)$

- ▶  $G$  : is a probabilistic key generation algorithm that takes no input but outputs the pair  $(vk, sk)$  namely verification key and secret key.
- ▶  $P$  : is an interactive algorithm called prover who takes input  $sk$
- ▶  $V$  : the verifier interactive algorithm who takes input the  $vk$

We require that when  $P(sk)$  interacts with  $V(vk)$  the output is always accept with probability 1.



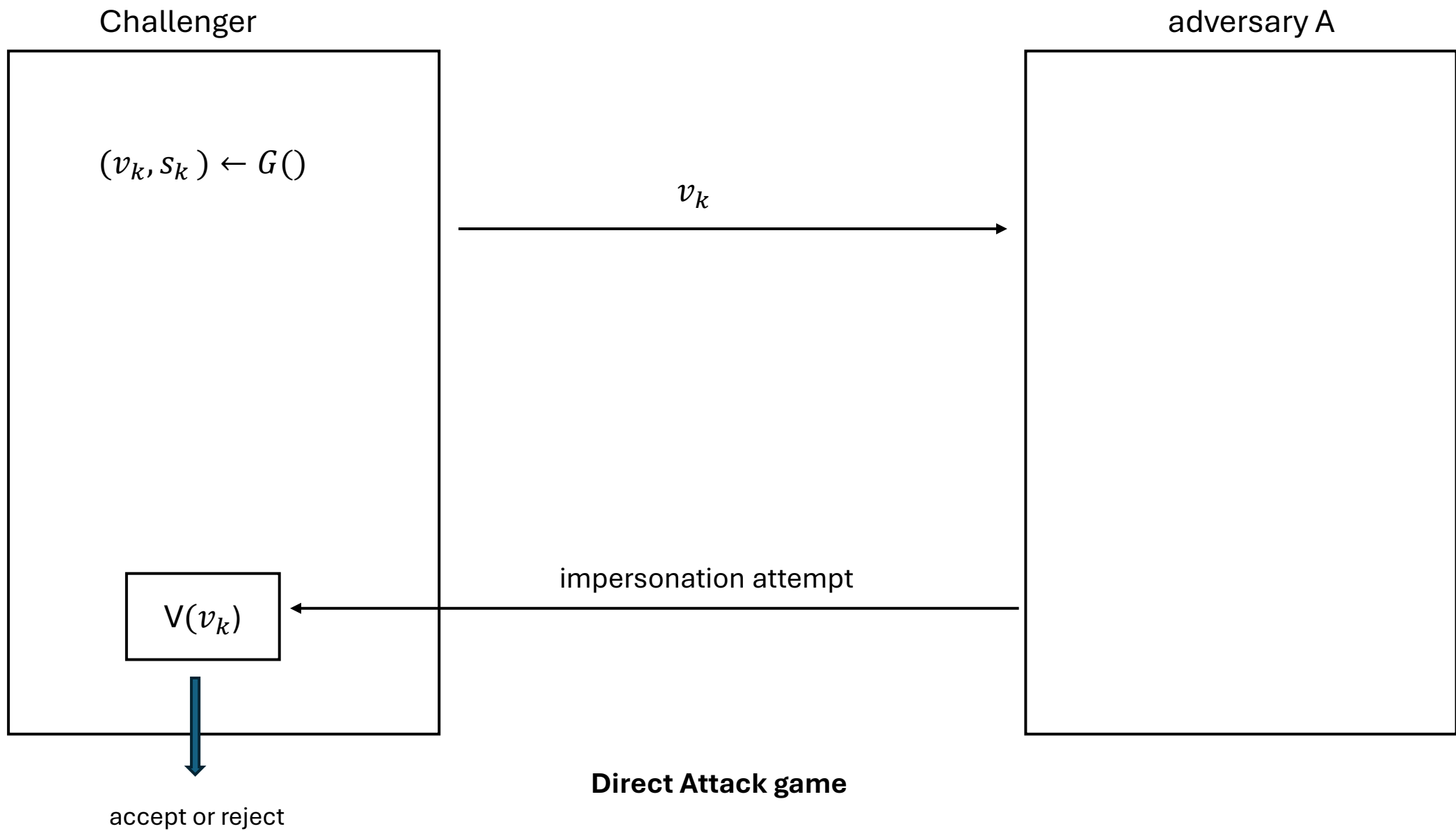
## Direct Attacks

**Attack Game: Secure identification against direct attacks:** For a given identification protocol  $I = (G, P, V)$  and given adversary  $A$

- ▶ Key generation phase: The challenger runs  $((vk, sk) \leftarrow G())$  and sends  $vk$  to  $A$ .
- ▶ Impersonation attempt: The challenger and  $A$  now interact where  $A$  tries to convince challenger(Verifier) to accept.

We say that adversary wins the game if  $V$  outputs accept at the end of the interaction.

In short this type of impersonation is clean and straight.





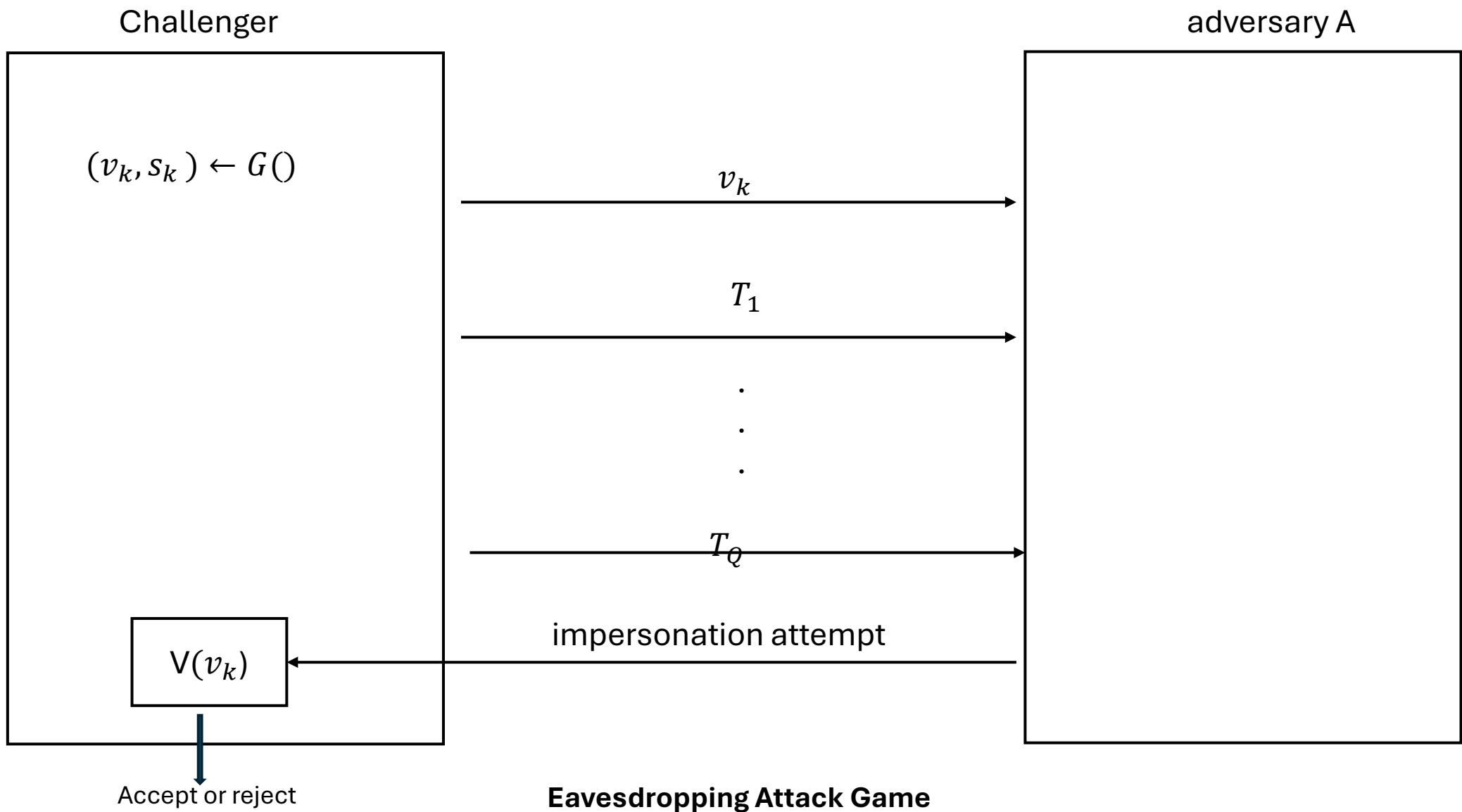
## Attack 2

### Attack Game: Secure ID against eavesdropping attack

- ▶ Key generation phase: The challenger runs  $((vk, sk) \leftarrow G())$  and sends  $vk$  to  $A$ .
- ▶ Eavesdrop Phase: The adversary request some number  $Q$ , of transcripts for the protocol to the challenger. Challenger then compiles send these transcripts  $T_1, T_2, \dots T_Q$  between prover and verifier to the adversary.
- ▶ Impersonation attempt: The challenger and  $A$  now interact where  $A$  tries to convince challenger(Verifier) to accept.

We say that adversary wins the game if  $V$  outputs accept at the end of the interaction.

In short this type of impersonation is same as direct attack but with a extra information to the adversary so he can start attacking using the information he got during eavesdrop phase.



## Challenge-Response Mechanism

Now we can define a general class of protocols called as **Sigma Protocols** which are based on challenge-response mechanism for secure identification.

## Challenge-Response Mechanism

Now we can define a general class of protocols called as **Sigma Protocols** which are based on challenge-response mechanism for secure identification.

**Effective Relation:** An effective relation is a binary relation  $R \subset X \times Y$  where  $X, Y, R$  are finite sets. Elements of  $Y$  are called as statements and if  $(x, y) \in R$  then  $x$  is called as the witness for  $y$ .

# Sigma Protocols

Definition (Sigma protocol). Let  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$  be an effective relation. A *Sigma protocol* for  $\mathcal{R}$  is a pair  $(P, V)$ .

# Sigma Protocols

Definition (Sigma protocol). Let  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$  be an effective relation. A *Sigma protocol* for  $\mathcal{R}$  is a pair  $(P, V)$ .

- ▶  $P$  is an interactive protocol algorithm called the *prover*, which takes as input a witness-statement pair  $(x, y) \in \mathcal{R}$ .

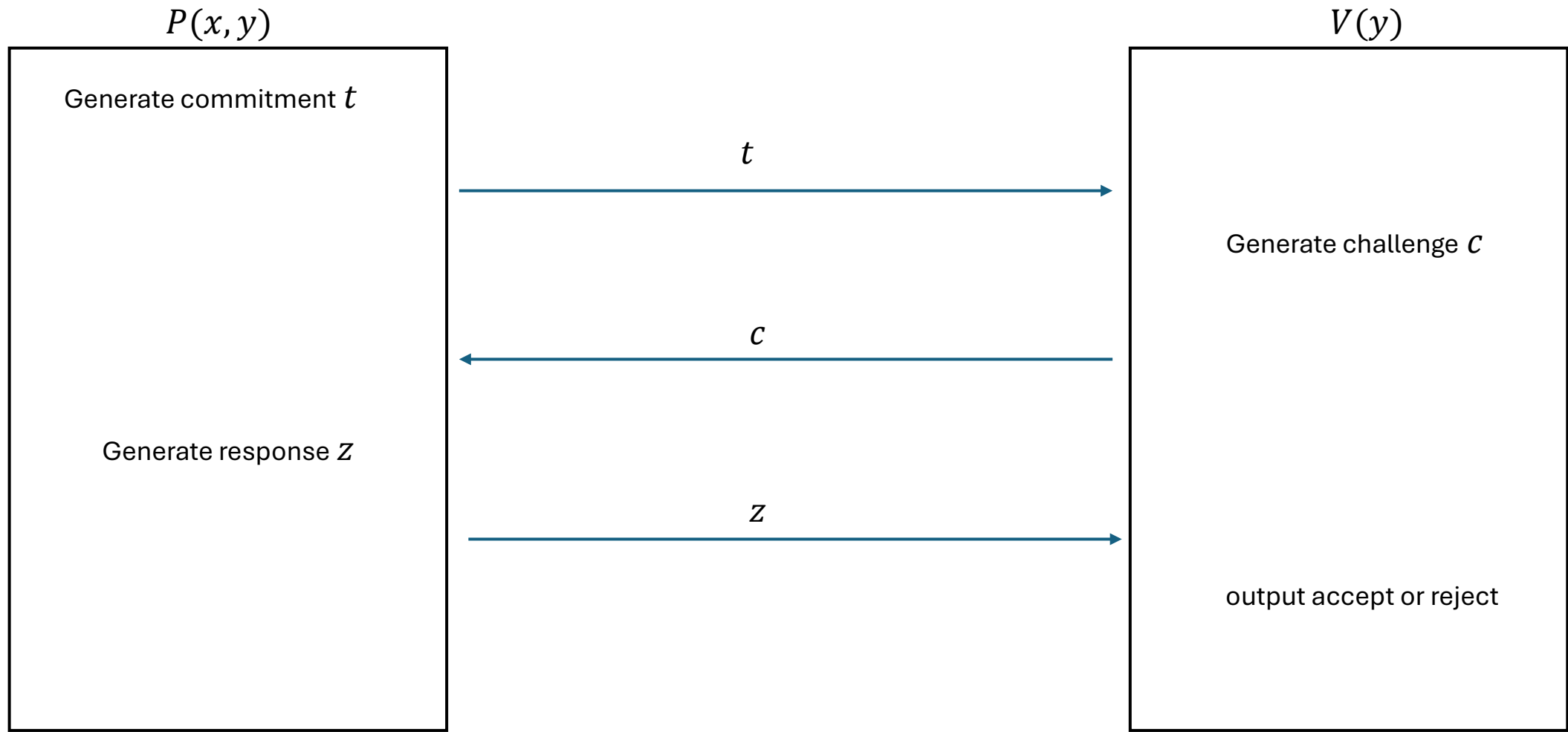
# Sigma Protocols

Definition (Sigma protocol). Let  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$  be an effective relation. A *Sigma protocol* for  $\mathcal{R}$  is a pair  $(P, V)$ .

- ▶  $P$  is an interactive protocol algorithm called the *prover*, which takes as input a witness-statement pair  $(x, y) \in \mathcal{R}$ .
- ▶  $V$  is an interactive protocol algorithm called the *verifier*, which takes as input a statement  $y \in \mathcal{Y}$ , and which outputs *accept* or *reject*.

- ▶  $P$  and  $V$  are structured so that an interaction between them always works as follows:
  - ▶ To start the protocol,  $P$  computes a message  $t$ , called the *commitment*, and sends  $t$  to  $V$ ;
  - ▶ Upon receiving  $P$ 's commitment  $t$ ,  $V$  chooses a *challenge*  $c$  at random from a finite *challenge space*  $\mathcal{C}$ , and sends  $c$  to  $P$ ;
  - ▶ Upon receiving  $V$ 's challenge  $c$ ,  $P$  computes a *response*  $z$ , and sends  $z$  to  $V$ ;
  - ▶ Upon receiving  $P$ 's response  $z$ ,  $V$  outputs either *accept* or *reject*, which must be computed strictly as a function of the statement  $y$  and the conversation  $(t, c, z)$ .





Execution of a Sigma Protocol

# Properties of Sigma Protocol

## 1. Completeness:

- ▶ For all  $(x, y) \in \mathcal{R}$ ,
- ▶ interaction between  $P(x, y)$  and  $V(y)$
- ▶ results in  $V(y)$  outputting *accept*.

# Properties

## 2. **Special Soundness:** (given sigma protocol)

- ▶ There exists an efficient deterministic algorithm

Ext (witness extractor)

# Properties

## 2. **Special Soundness:** (given sigma protocol)

- ▶ There exists an efficient deterministic algorithm

Ext (witness extractor)

- ▶ Given:

a statement  $y \in \mathcal{Y}$ ,  
two accepting transcripts  $(t, c, z)$  and  $(t, c', z')$ ,  
with  $c \neq c'$ ,

# Properties

## 2. **Special Soundness:** (given sigma protocol)

- ▶ There exists an efficient deterministic algorithm

Ext (witness extractor)

- ▶ Given:

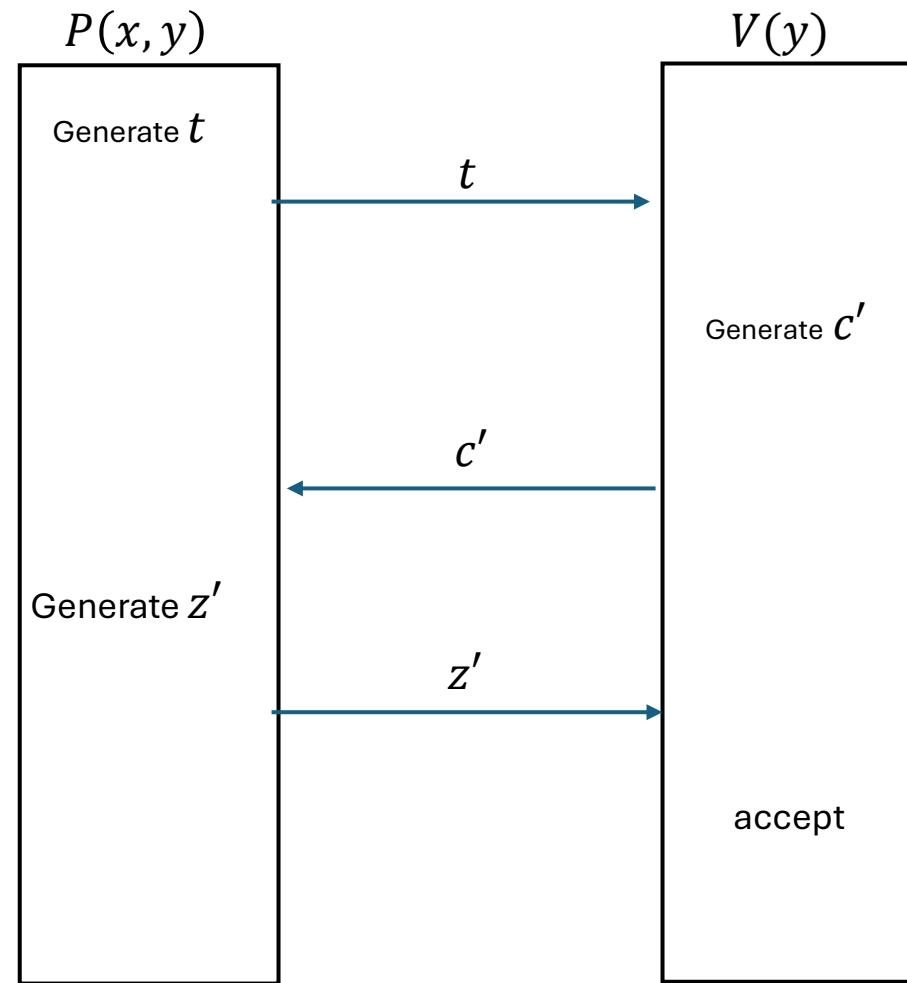
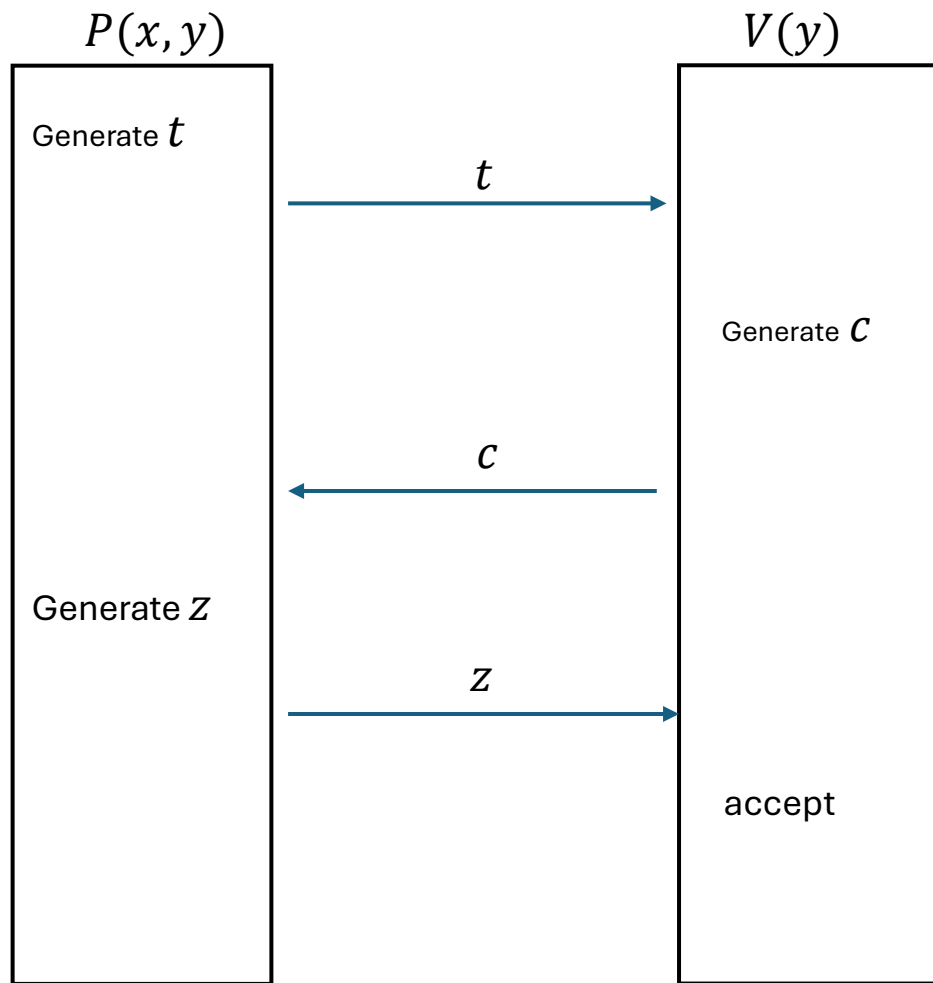
a statement  $y \in \mathcal{Y}$ ,

two accepting transcripts  $(t, c, z)$  and  $(t, c', z')$ ,

with  $c \neq c'$ ,

Ext outputs  $x \in \mathcal{X}$  such that

$$(x, y) \in \mathcal{R}.$$



Special Soundness

# HVZK

**Honest Verifier Zero Knowledge (HVZK)** If there exist an efficient algorithm **Sim**:

(i) for all  $y \in \mathcal{Y}$ , algorithm **Sim** always outputs an accepting conversation  $(t, c, z)$ ;

(ii) for all  $(x, y) \in \mathcal{R}$ ,

$$\{P(x, y), V(y)\} \approx \{\text{Sim}(y)\}$$

## Special HVZK

### Special Honest Verifier Zero Knowledge (Special HVZK)

If there exists an efficient probabilistic algorithm **Sim** such that:

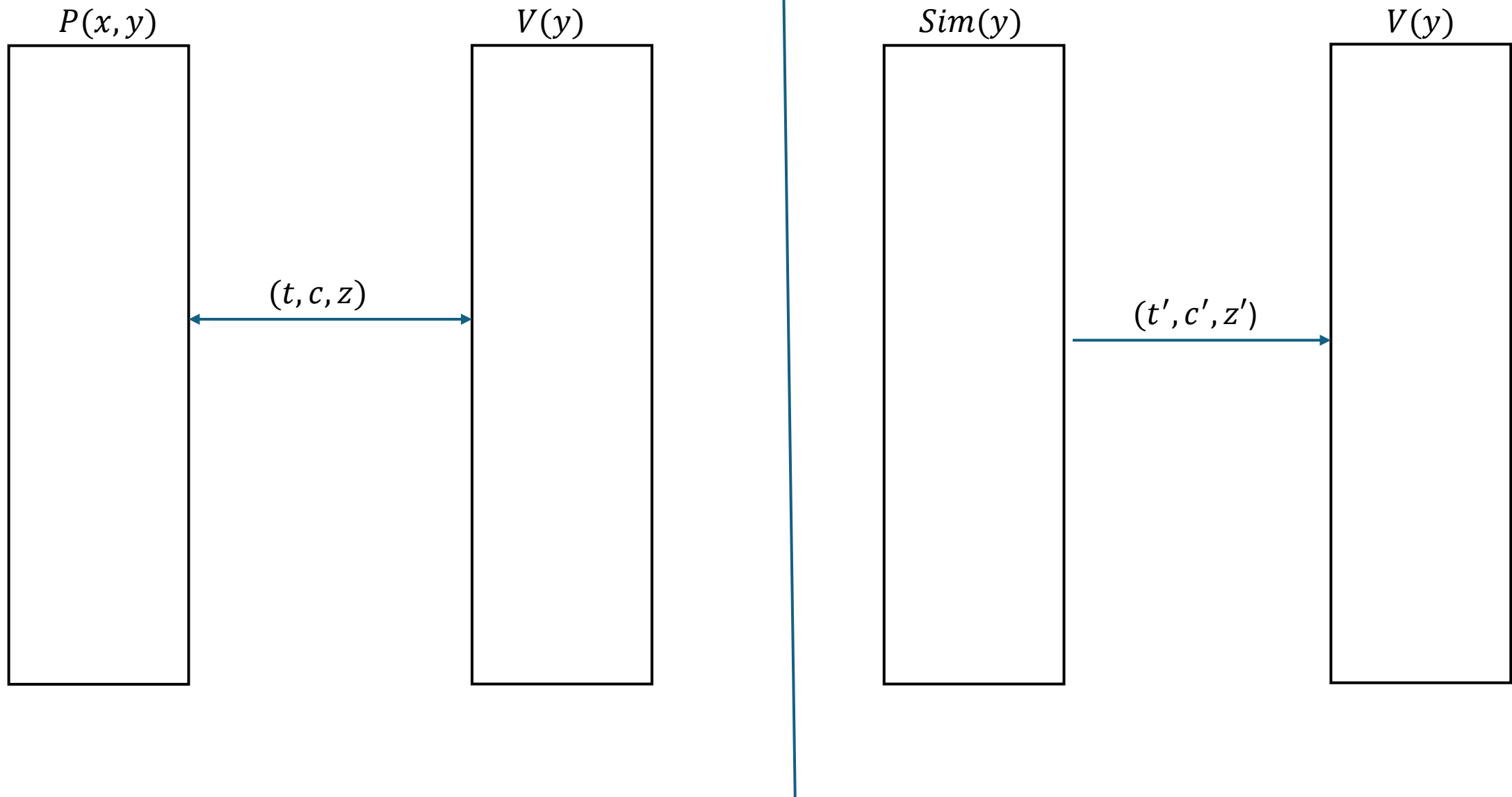
(i) for all  $(y, c) \in \mathcal{Y} \times \mathcal{C}$ , algorithm **Sim** always outputs an accepting conversation  $(t, z)$ ;

(ii) for all  $(x, y) \in \mathcal{R}$ ,

$$\{P(x, y), V(y)\} \approx \{\text{Sim}(y, c)\}$$

This is a special case of HVZK where the simulator takes both  $y$  and  $c$  as inputs.





HVZK Property: The distributions  $\{P(x, y), V(y)\} \approx \{Sim(y), V(y)\}$  are *Computationally Indistinguishable*.

## Shnorr's ID protocol

►  $\mathbf{R} = \{(\alpha, g^\alpha) \mid \alpha \in \mathbb{Z}_q, g^\alpha \in \mathbb{G}\}$  (Discrete Log Relation)

## Shnorr's ID protocol

- ▶  $\mathbf{R} = \{(\alpha, g^\alpha) \mid \alpha \in \mathbb{Z}_q, g^\alpha \in \mathbb{G}\}$  (Discrete Log Relation)
- ▶ Keygen():  $G$  is a cyclic group of prime order.  $\alpha \in \mathbb{Z}_q, g^\alpha \in G$   
 $(sk, vk) \leftarrow (\alpha, g^\alpha)$

## Shnorr's ID protocol

- ▶  $\mathbf{R} = \{(\alpha, g^\alpha) \mid \alpha \in \mathbb{Z}_q, g^\alpha \in \mathbb{G}\}$  (Discrete Log Relation)
- ▶ Keygen():  $G$  is a cyclic group of prime order.  $\alpha \in \mathbb{Z}_q, g^\alpha \in G$   
 $(sk, vk) \leftarrow (\alpha, g^\alpha)$
- ▶ The protocol between  $P(\alpha)$  and  $V(g^\alpha)$ 
  1.  $P$  computes  $\alpha_t \leftarrow \mathbb{Z}_q, u_t \leftarrow g^{\alpha_t}$ , and send  $u_t$  to  $V$ .
  2.  $V$  computes  $c \leftarrow C$  and send to  $P$ .  $C \subset \mathbb{Z}_q$  called as challenge space.
  3.  $P$  computes  $\alpha_z = \alpha_t + \alpha \cdot c$  and send to  $V$ .
  4.  $V$  checks if  $g^{\alpha_z} = u_t \cdot (vk)^c$

$P(\alpha, g^\alpha)$

$$\alpha_t \leftarrow \mathbb{Z}_q, \quad u_t \leftarrow g^{\alpha_t}$$

$$\alpha_z \leftarrow \alpha_t + \alpha \cdot c$$

$V(g^\alpha)$

$$c \leftarrow \mathcal{C}$$

$$g^{\alpha_z} = u_t \cdot u^c$$

$u_t$

$c$

$\alpha_z$

Schnorr's Identification Protocol

Theorem : **Schnorr ID protocol has special soundness Property.**

▶ Given  $(u_t, c_1, \alpha_z)$  &  $(u_t, c_2, \alpha'_z)$  we can extract witness.

▶  $\Delta_z = \alpha_z - \alpha'_z = (\alpha_t - \alpha_t) + \alpha(c_1 - c_2)$

▶  $\alpha = \Delta_z / (c_1 - c_2)$

Theorem : **Schnorr's identification protocol is HVZK.**

*Proof.* On input  $vk = u$ , the simulator Sim computes (in reverse order):

$$\alpha_z \xleftarrow{\$} \mathbb{Z}_q, \quad c \xleftarrow{\$} \mathcal{C}, \quad u_t \leftarrow g^{\alpha_z} / u^c,$$

and outputs the conversation  $(u_t, c, \alpha_z)$ .

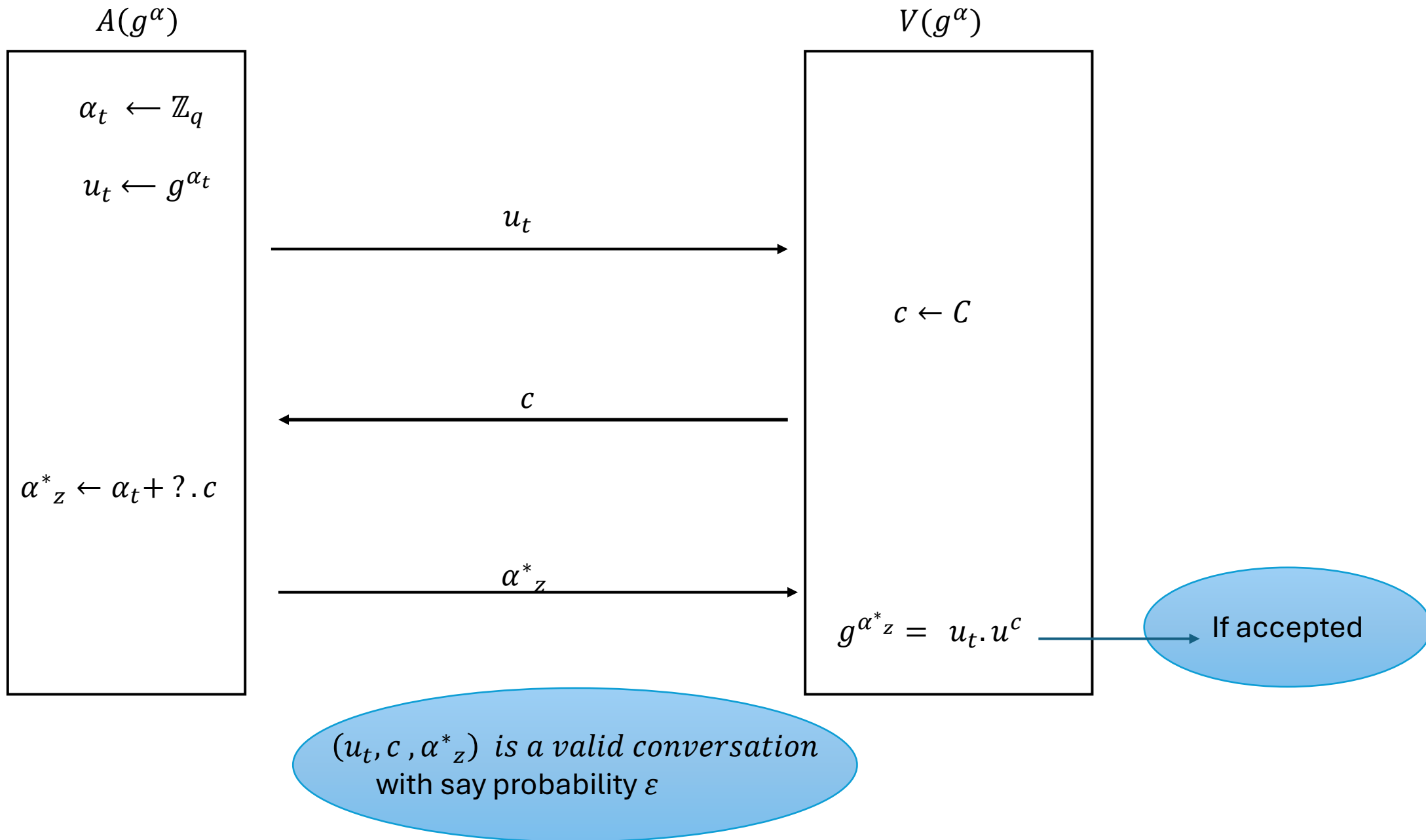
It should be clear that this is the same as the output distribution of the real conversation.

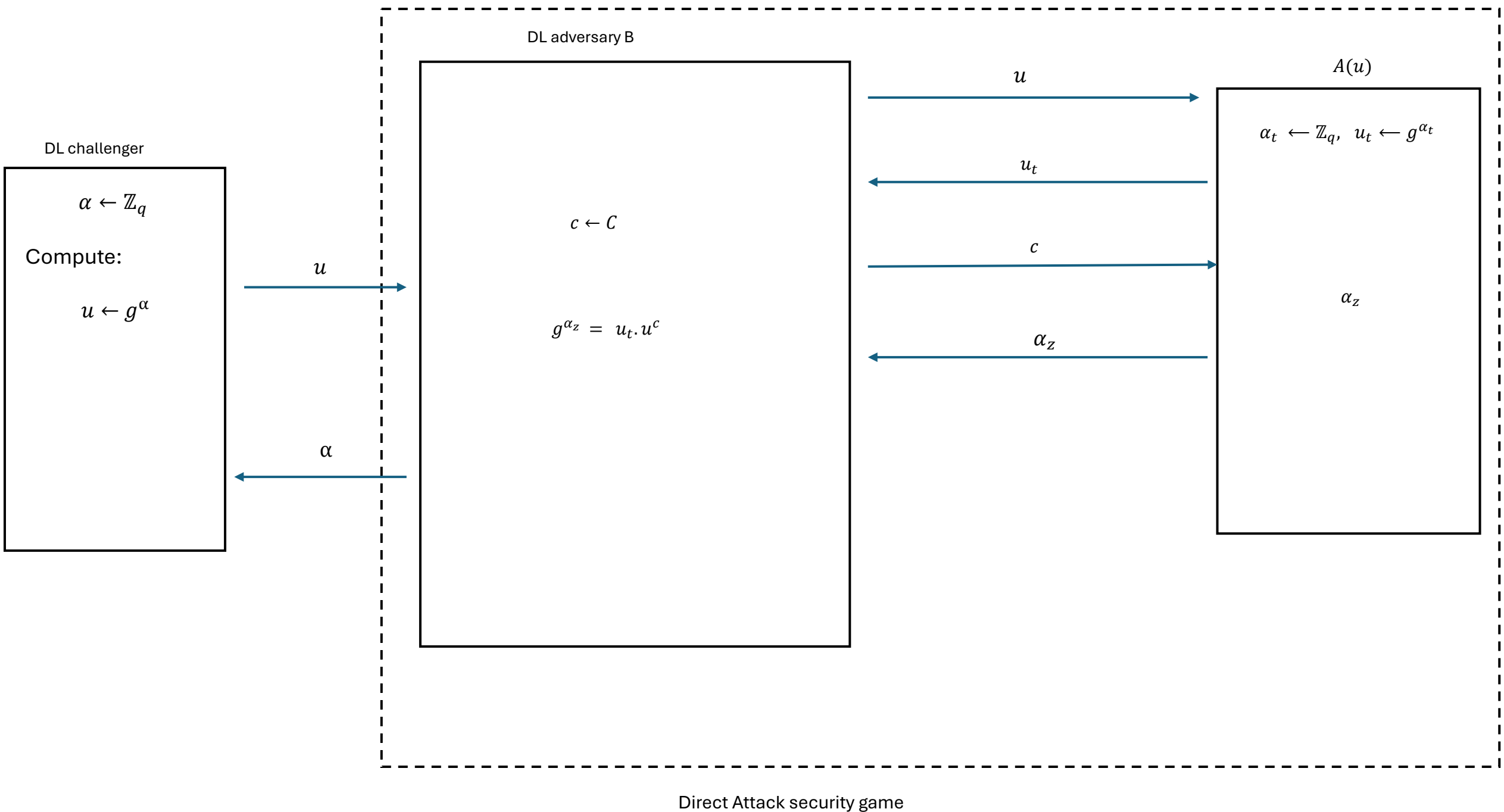
## Direct attack security

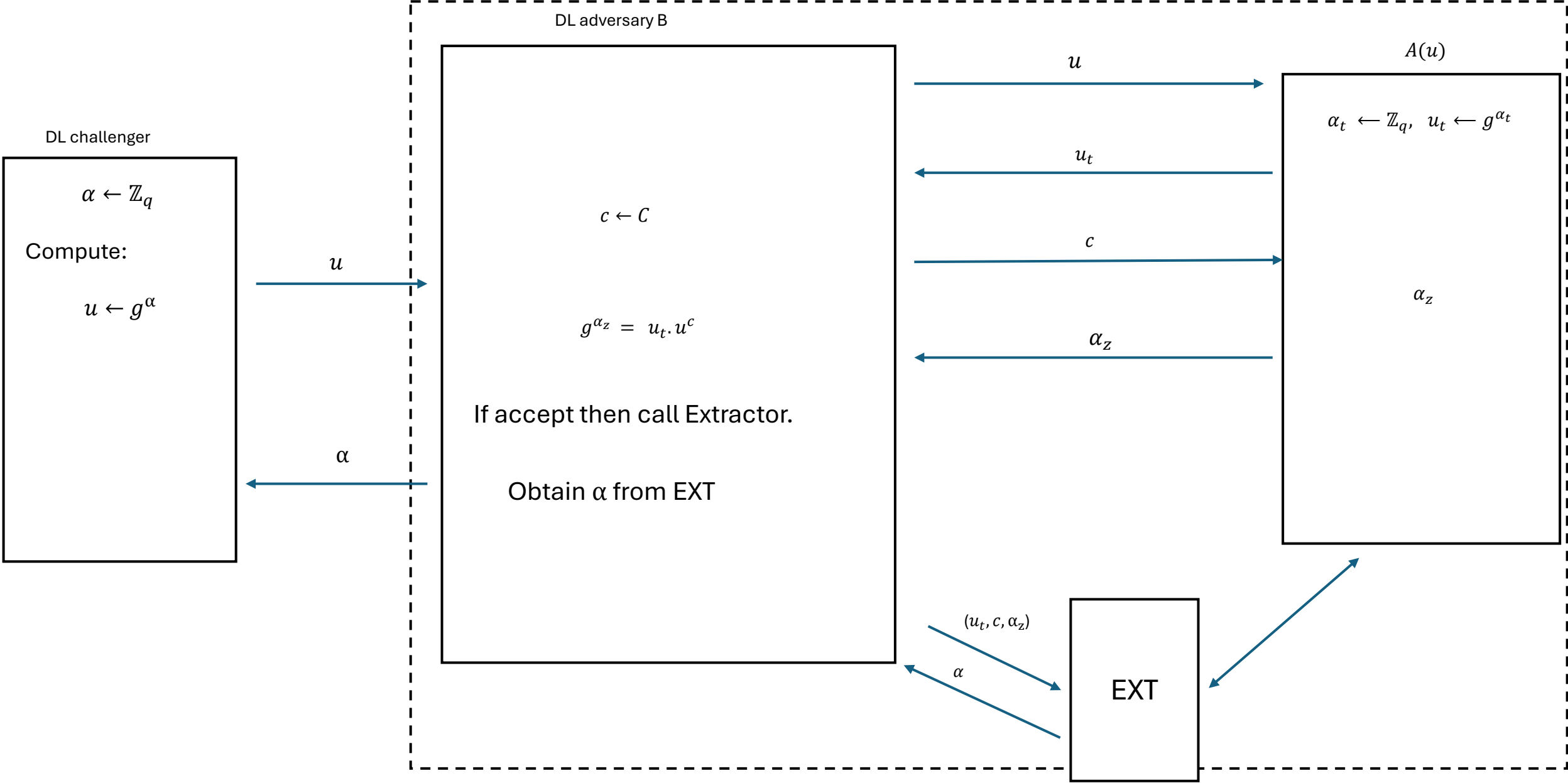
Theorem: Schnorr ID protocol is secure against direct attack if DL is hard for  $G$  and  $N = |C|$  is super-polynomial in size.

Proof: We know that Schnorr ID protocol has special soundness property. We can build a reduction to DL instance.

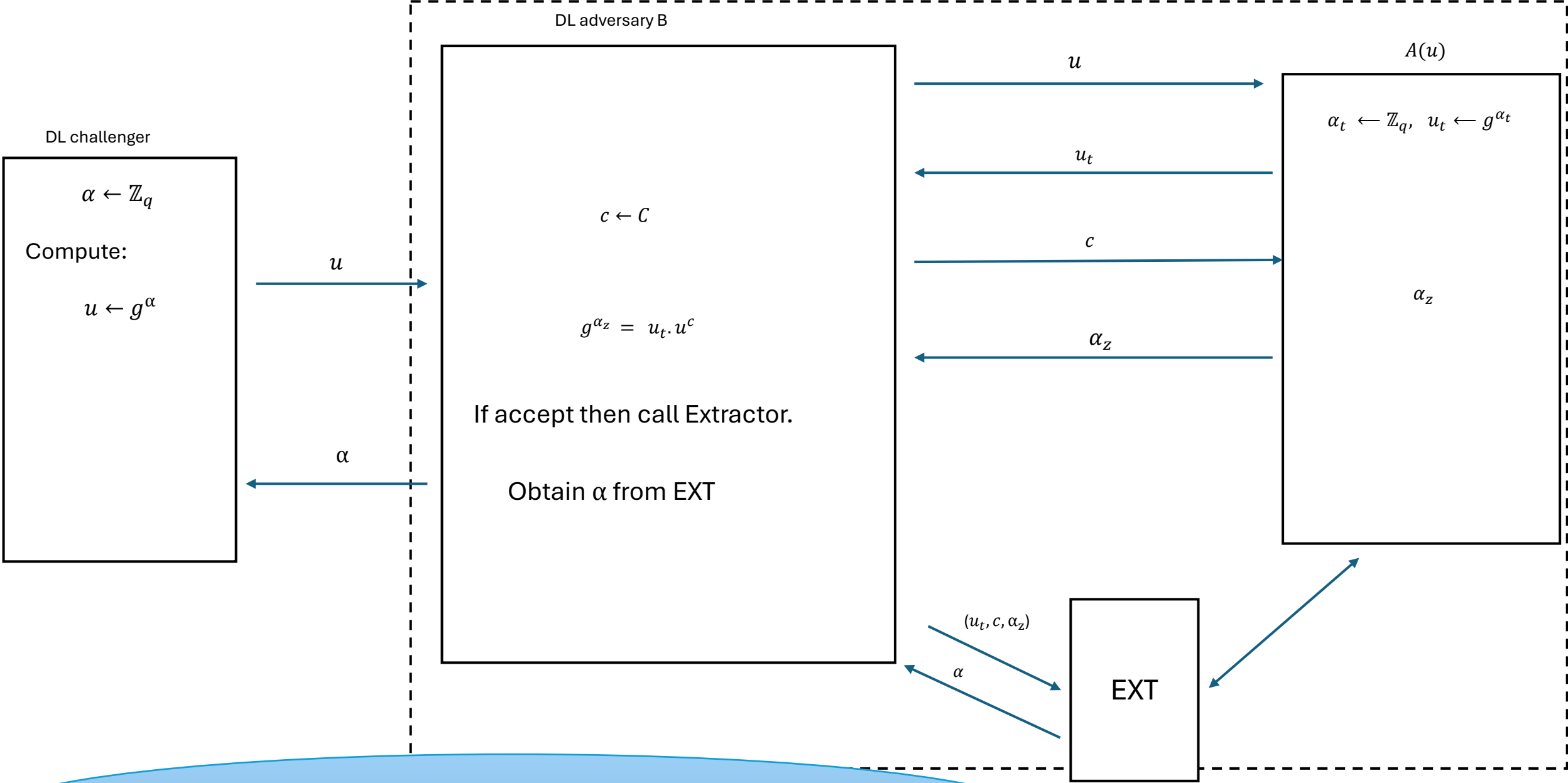




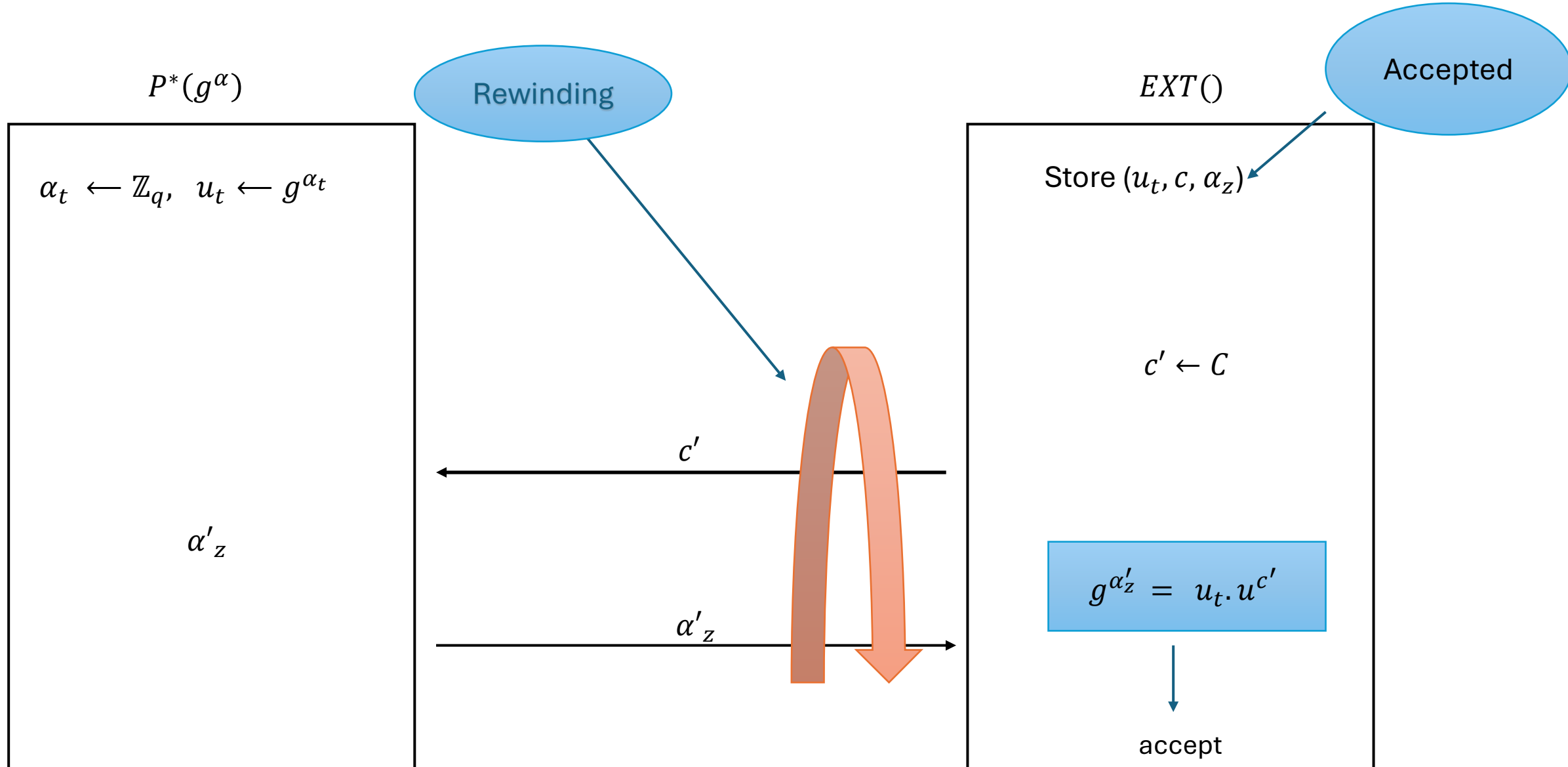




Direct Attack security game



If A succeeds with Probability  $\varepsilon$  then B succeeds with Probability  $\varepsilon^2$

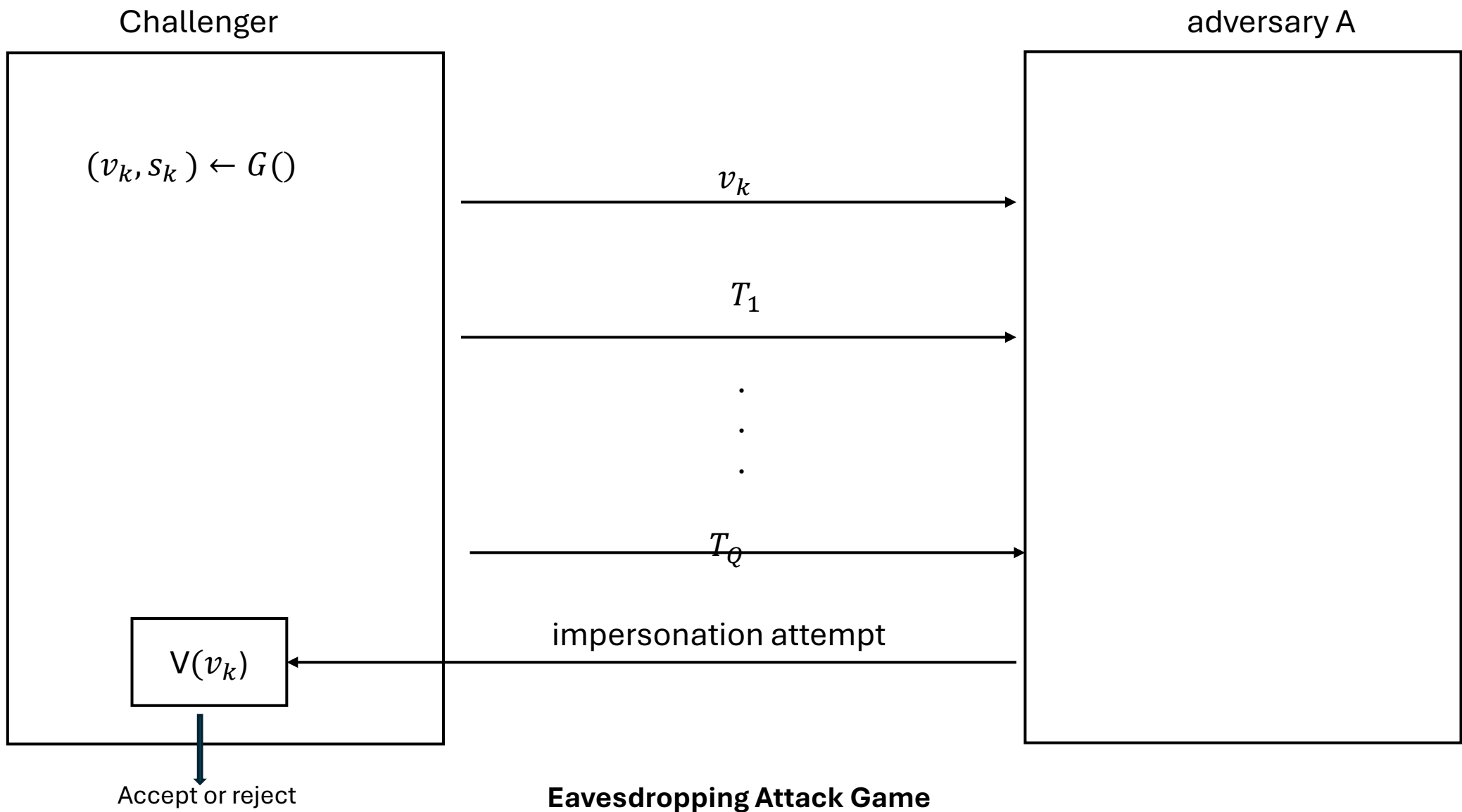


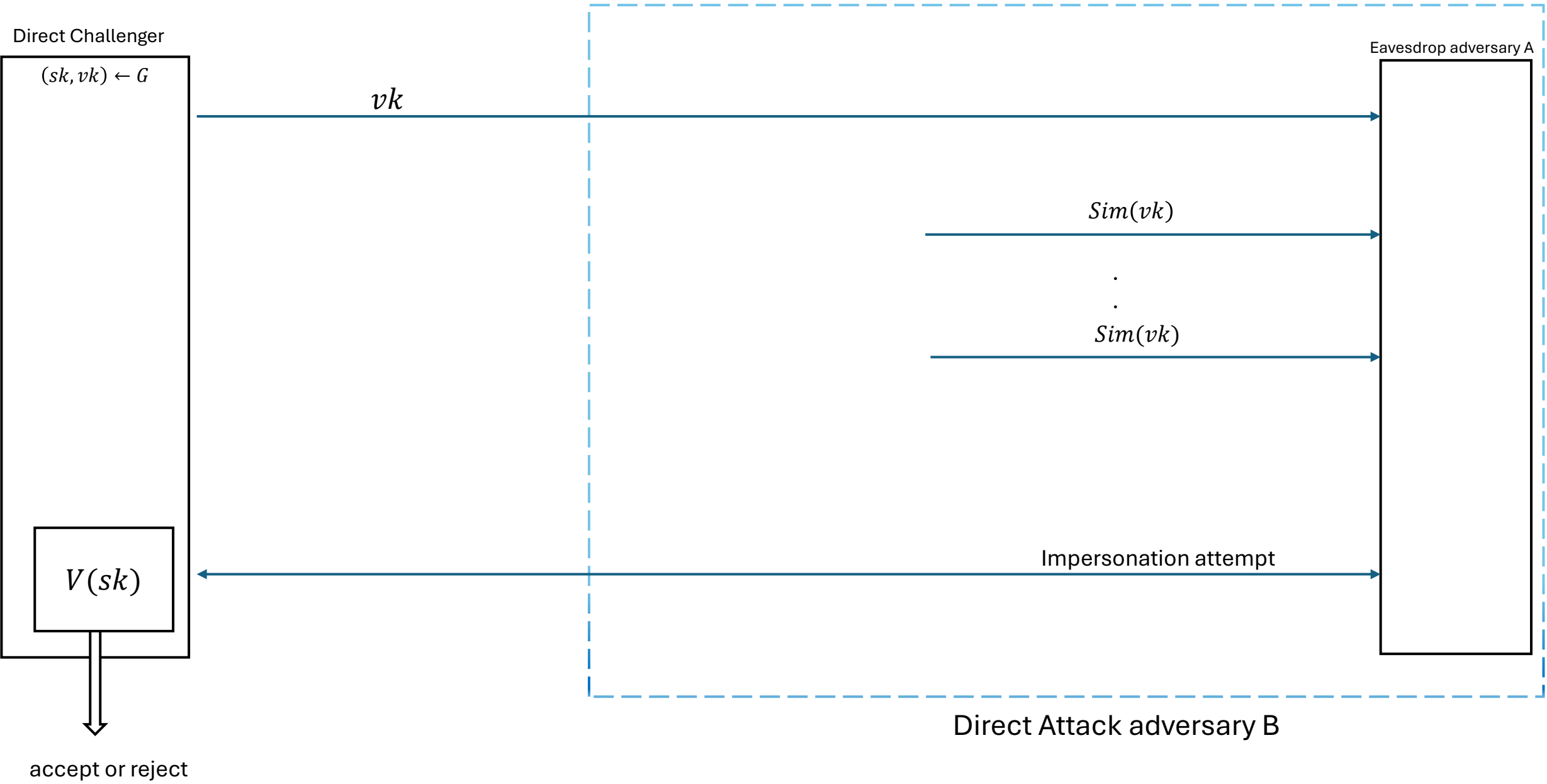
Rewinding Power of Extractor

## Eavesdropping security

Theorem: Schnorr ID protocol is secure against Direct attacks then it is also secure against eavesdropping attack.

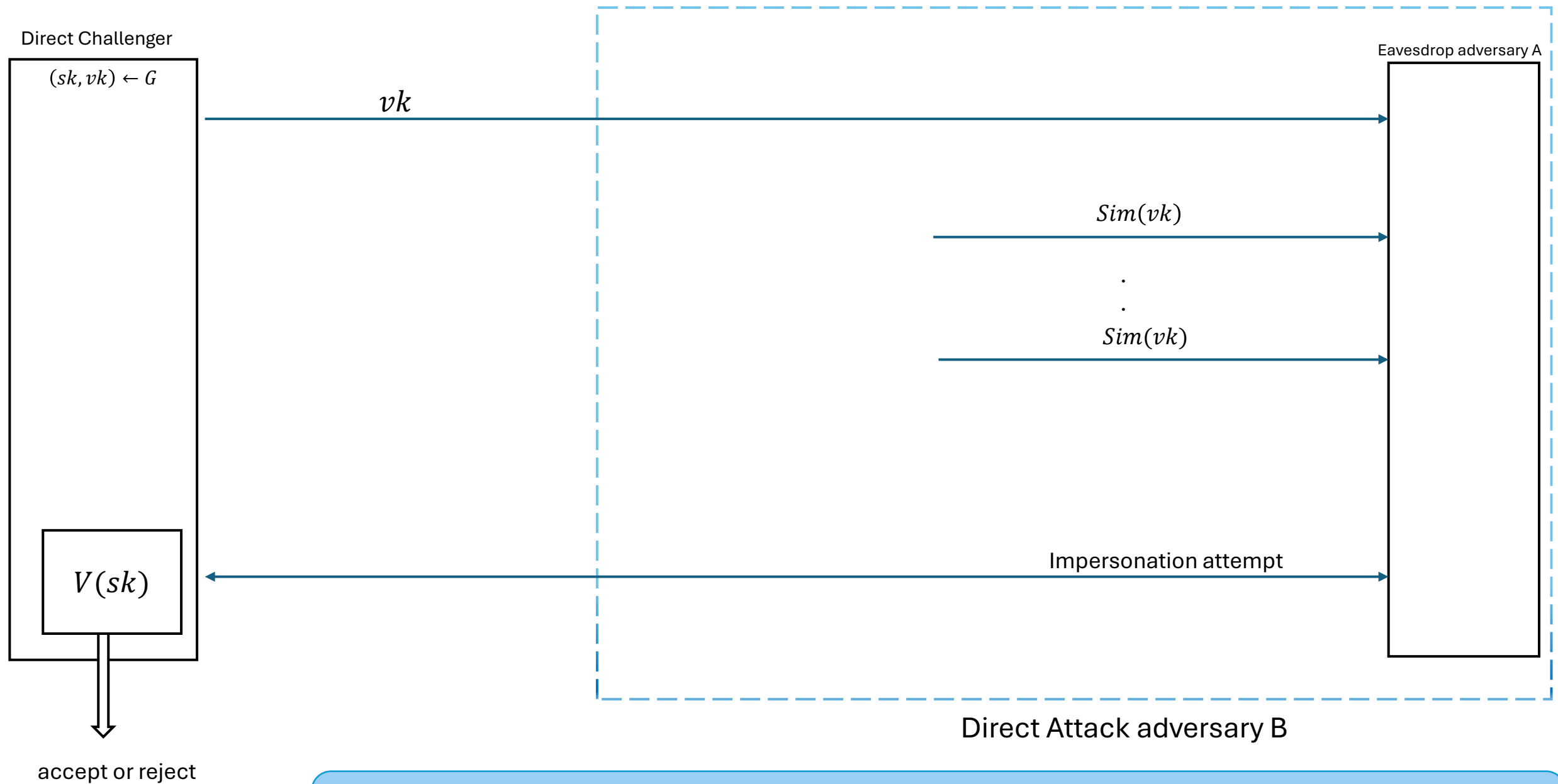
We will use the fact that Schnorr protocol is HVZK.





Reduction to Direct attack game





Probability of B succeeds in Direct Attack = Probability of A succeeds in Eavesdrop Attack

## Quick Recap

- ▶ We have seen above the powerful class of secure identification ” **sigma protocols**”
- ▶ Crucially with 2 key properties:
  - ▶ Special Soundness (Verifier Security)
  - ▶ Honest Verifier Zero Knowledge (Prover Security)

## Quick Recap

- ▶ We have seen above the powerful class of secure identification "**sigma protocols**"
- ▶ Crucially with 2 key properties:
  - ▶ Special Soundness (Verifier Security)
  - ▶ Honest Verifier Zero Knowledge (Prover Security)

**Can we realize above notion with no-interaction ?**

# Non-interactive

Can we build Non-interactive ID protocols?

Applications:

- ▶ Signatures
- ▶ Blockchains

# Non-interactive

Can we build Non-interactive ID protocols?

Applications:

- ▶ Signatures
- ▶ Blockchains

First positive answer by **Amos Fiat** & **Adi Shamir** in 1986

# From Sigma Protocols to Fiat-Shamir

- ▶ Sigma protocols are **interactive by design**:

# From Sigma Protocols to Fiat-Shamir

- ▶ Sigma protocols are **interactive by design**:
- ▶ To make Sigma protocols **non-interactive**:
  - ▶ Prover must **simulate verifier's challenge** without cheating

## What Fiat–Shamir Is Replacing

- ▶ In a Sigma protocol, soundness and extraction rely on one key fact:  
*The challenge is **fresh, unpredictable, and independent** of the prover's commitment.*
- ▶ This unpredictability is what enables **special soundness**.



$P(x, y)$

Generate commitment  $t$

*Compute*  $H(y, t)$

$c \leftarrow H(y, t)$

Generate response  $Z$

$(t, z)$

$V(y)$

Compute:

$H(y, t) \rightarrow c$

Verify the full transcript:

$(t, c, z)$

output accept or reject

Fiat shamir Transformation

## What We Need from the Hash Function

- ▶ Fiat–Shamir requires the hash function to act as an **honest verifier**.
- ▶ Required properties:
  - ▶ Uniform challenge sampling:  $H(t, y) \sim \mathcal{C}$ ,
  - ▶ Unpredictability before commitment,
- ▶ Only the Random Oracle Model provides all three simultaneously.

# Core Insight

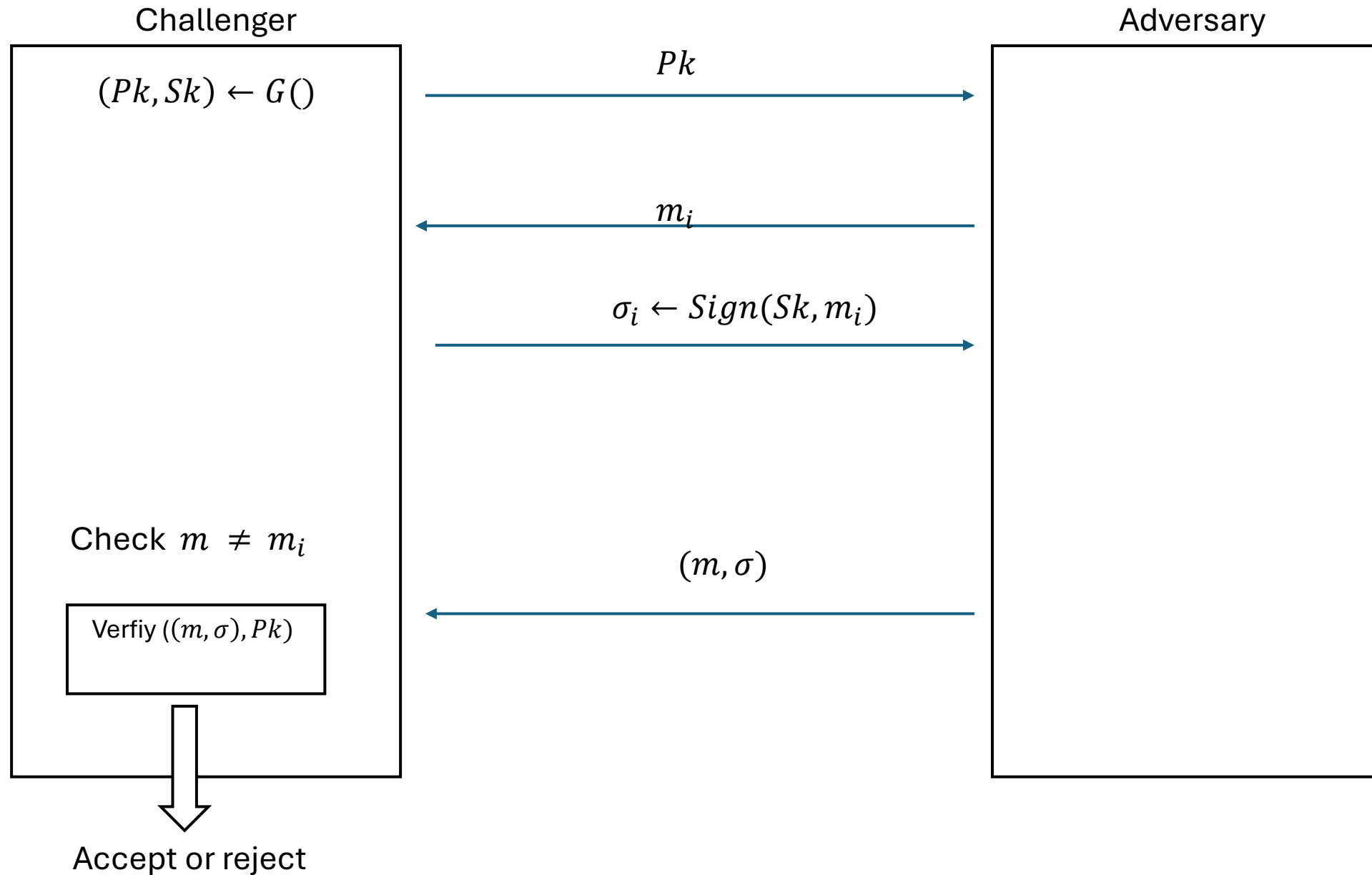
Fiat shamir transform:

- ▶ Provably secure in the ROM.
- ▶ No general security proof in the standard model.

# Digital Signatures

Quickly, Digital signature scheme  $Sig = (G, S, V)$  as follows:

- ▶  $G() \rightarrow (s_k, p_k)$
- ▶  $S(s_k, m) \rightarrow \sigma$
- ▶  $V(p_k, m, \sigma) \rightarrow \text{accept/reject}$



Digital Signature Security Game

## FS signature

- ▶ a sigma protocol  $(P,V)$  for a relation  $R$  we assume that the conversations are of the form  $(t, c, z)$  where  $t \in T$   $c \in C$   $z \in Z$
- ▶ a key generation algorithm  $G$  for  $R$ ;
- ▶ a hash function  $H : M \times T \rightarrow C$  which will be modeled as a random oracle; the set  $M$  will be the message space of the signature scheme.

The Fiat shamir signature scheme derived from  $G$  and  $(P,V)$  works as follows:

- ▶ The key generation algorithm is  $G$ , so public key  $Pk = y$ , where  $y \in Y$  and a secret key is of the form  $sk = (x, y) \in R$
- ▶ To sign a message  $m \in M$  using a secret key  $sk = (x, y)$ , the signing algorithm runs as follows:
  - ▶ it starts the prover  $P(x, y)$ , obtaining the commitment  $t \in T$
  - ▶ It computes a challenge  $c \leftarrow H(m, t)$
  - ▶ finally it feeds  $c$  to the prover to obtain the response  $z$  and outputs the signature  $\sigma = (t, z) \in T \times Z$
  - ▶ Verification of signature: On input  $(m, t, z)$  verifier do  $H(m, t) \rightarrow c$  check if whether  $(t, c, z)$  is an accepting conversation for  $y$ .

$P(x, y)$

$Sk = x, Pk = y$

*commitment*  
 $t \leftarrow T$

$c \leftarrow H(m, t)$

$m \leftarrow M$

*Compute*  
 $z$

$(m, \sigma = (t, z))$

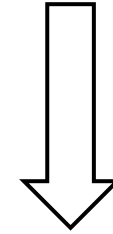
$V(y)$

$H(m, t) \rightarrow c$

*Verify* $(t, c, z)$  is valid

*conversation or not*

*Verify* $(m, \sigma)$



output accept or reject

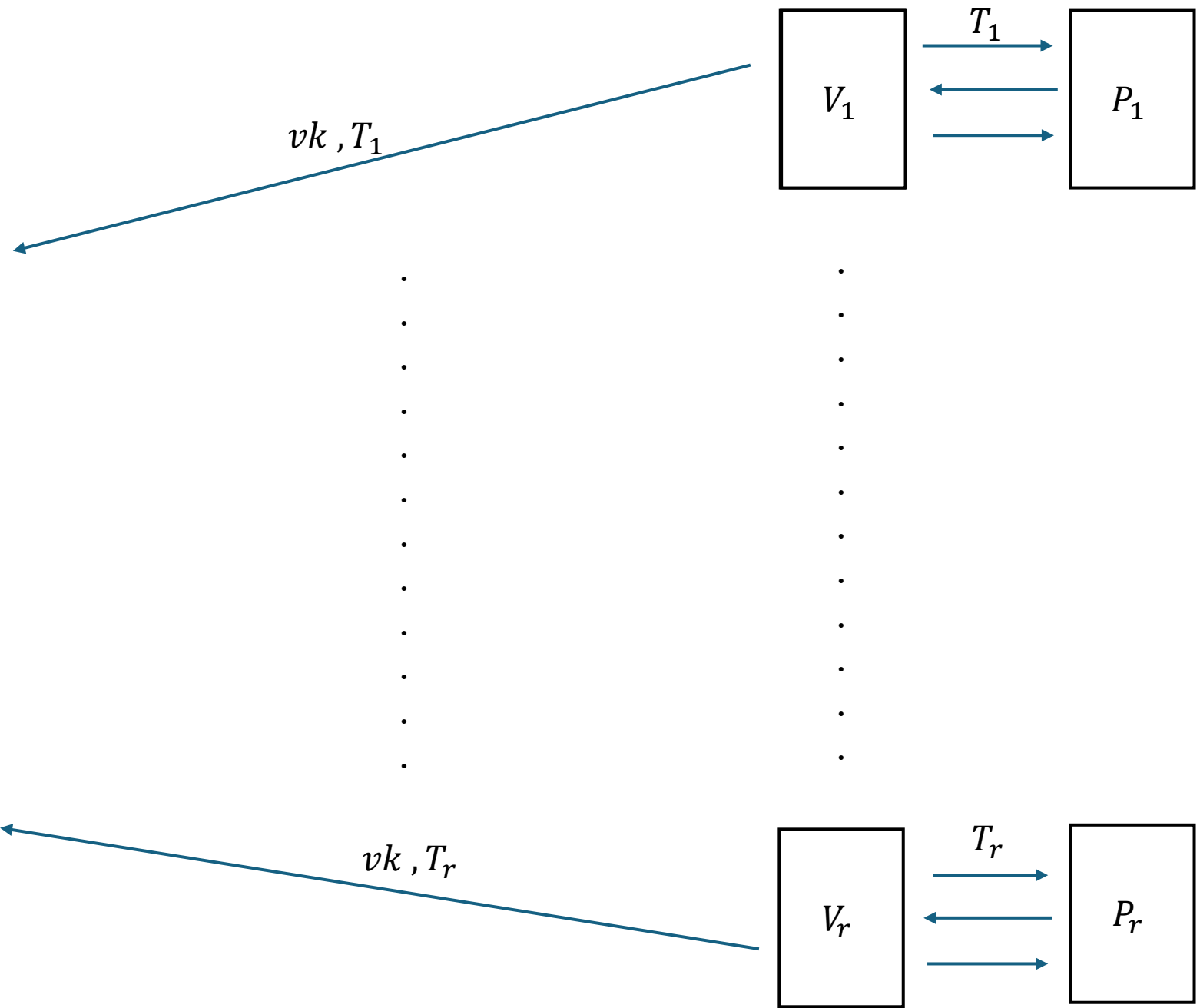
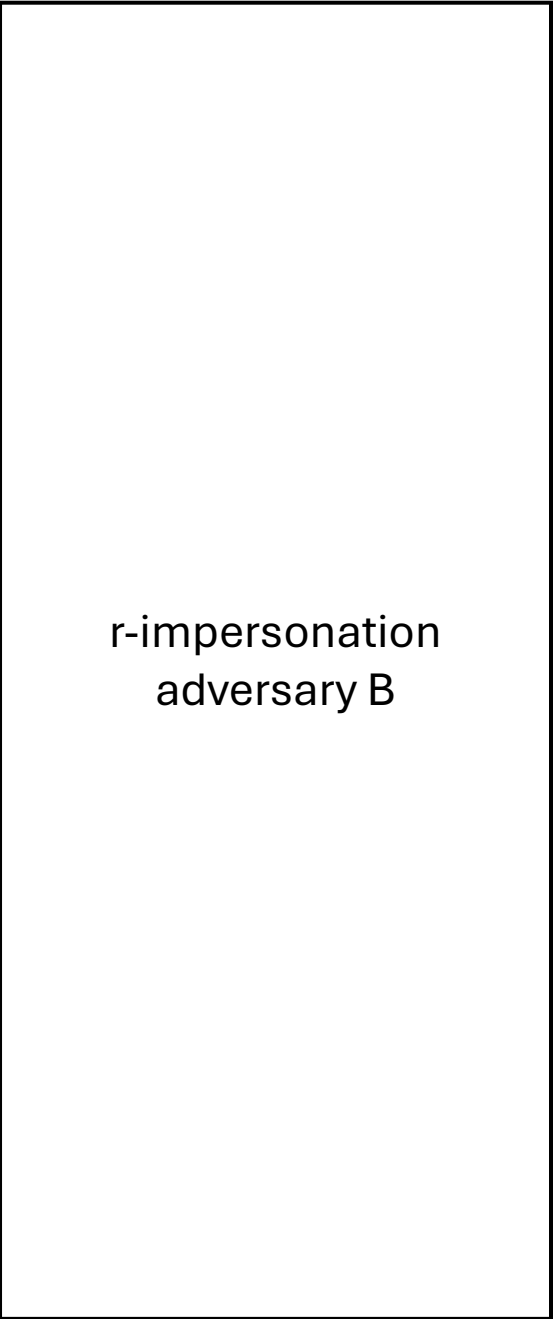
Fiat Shamir signature

## FS security

**Theorem:** If  $H$  is modeled as a random oracle, the identification scheme  $\mathcal{I} = (G, P, V)$  is secure against eavesdropping attacks, and  $(P, V)$  has unpredictable commitments, then the Fiat–Shamir signature scheme  $\mathcal{S}$  derived from  $G$  and  $(P, V)$  is secure.



**r-impersonation eavesdropping attack game**



## Lemma

**Lemma.** For every  $r$ -impersonation eavesdropping adversary  $A$ , there exists a standard eavesdropping adversary  $B$  (an elementary wrapper around  $A$ ) such that

$$rID2adv[A, I, r] \leq r \cdot ID2adv[B, I].$$

## Restating FS security for shcnorr Signature

**Theorem:** If  $H$  is modeled as a random oracle, the identification scheme  $\mathcal{I}_{Sch} = (G, P, V)$  is secure against r-impersonation eavesdropping attacks, and  $(P, V)$  has unpredictable commitments, then the Fiat–Shamir shcnorr signature scheme  $\mathcal{S}$  derived from  $G$  and  $(P, V)$  is secure.

$$P(\alpha, g^\alpha)$$

$$\alpha_t \leftarrow \mathbb{Z}_q, \quad u_t \leftarrow g^{\alpha_t}$$

$$H(m, u_t) \rightarrow c$$

$$\alpha_z \leftarrow \alpha_t + \alpha \cdot c$$

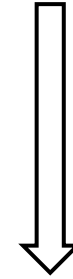
$$(m, \sigma = (u_t, \alpha_z))$$

$$V(g^\alpha)$$

$$H(m, u_t) \rightarrow c$$

Verify

$$g^{\alpha_z} = u_t \cdot u^c$$



Accept or reject

Schnorr's signature Scheme

## Game 0

- ▶ Game 0: Original signature security game with random oracle queries.
- ▶ Simplified assumption: Adversaries forgery attempt must be went through random oracle.
- ▶  $W_0$  = The event where adversary wins the Game 0

$$\alpha \leftarrow \mathbb{Z}_q, u \leftarrow g^\alpha$$

Initialize:  $RO : M \times G \rightarrow C$  ,  $Explicit : M \times G \rightarrow \mathbb{Z}$

Send the public key  $u$  to A

1) Upon receiving the  $i$ -th signing query  $m_i \in M$ :

$$\alpha_{ti} \leftarrow \mathbb{Z}_q, u_{ti} \leftarrow g^{\alpha_{ti}}, c_i \leftarrow C$$

\*  $if (m_i, u_{ti}) \in Domain(RO): c_i \leftarrow RO[m_i, u_{ti}]$

$$RO[m_i, u_{ti}] \leftarrow c_i$$

$$\alpha_{zi} \leftarrow \alpha_{ti} + \alpha \cdot c_i$$

Send  $(u_{ti}, \alpha_{zi})$  to A

2) Upon receiving  $j$ -th RO query  $(\widehat{m}_j, \widehat{u}_j) \in M \times G$

if  $(\widehat{m}_j, \widehat{u}_j) \notin Domain(RO)$  then

$$RO[(\widehat{m}_j, \widehat{u}_j)] \leftarrow C, Explicit[(\widehat{m}_j, \widehat{u}_j)] \leftarrow j$$

send  $RO[(\widehat{m}_j, \widehat{u}_j)]$  to A

3) Upon receiving a forgery attempt  $(m, u_t, \alpha_z)$ :

if  $g^{\alpha_z} = u_t \cdot u^c$  where  $c = RO[m, u_t]$

then output **win**

else output **lose**

Prepare set of signing queries:

$$\{m_i \mid i = 1, \dots, Q_s\}$$

Prepare set of RO queries:

$$(\widehat{m}_j, \widehat{u}_j)$$

Game 0

## Game 0 Analysis

- ▶ Consistency check for previous RO queries.
- ▶  $SIG^{roadv}[A, S_{Sch}] = Pr[W_0]$



$$\alpha \leftarrow \mathbb{Z}_q, u \leftarrow g^\alpha$$

Initialize:  $RO : M \times G \rightarrow C$  ,  $Explicit : M \times G \rightarrow \mathbb{Z}$

Send the public key  $u$  to A

1) Upon receiving the  $i$ -th signing query  $m_i \in M$ :

$$\alpha_{ti} \leftarrow \mathbb{Z}_q, u_{ti} \leftarrow g^{\alpha_{ti}}, c_i \leftarrow C$$

\*

$$RO[m_i, u_{ti}] \leftarrow c_i$$

$$\alpha_{zi} \leftarrow \alpha_{ti} + \alpha \cdot c_i$$

Send  $(u_{ti}, \alpha_{zi})$  to A

2) Upon receiving  $j$ -th RO query  $(\widehat{m}_j, \widehat{u}_j) \in M \times G$

if  $(\widehat{m}_j, \widehat{u}_j) \notin \text{Domain}(RO)$  then

$$RO[(\widehat{m}_j, \widehat{u}_j)] \leftarrow C, \text{Explicit}[(\widehat{m}_j, \widehat{u}_j)] \leftarrow j$$

send  $RO[(\widehat{m}_j, \widehat{u}_j)]$  to A

3) Upon receiving a forgery attempt  $(m, u_t, \alpha_z)$ :

if  $g^{\alpha_z} = u_t \cdot u^c$  where  $c = RO[m, u_t]$

then output **win**

else output **lose**

Prepare set of signing queries:

$$\{m_i \mid i = 1, \dots, Q_s\}$$

Prepare set of RO queries:

$$(\widehat{m}_j, \widehat{u}_j)$$

Game 1

## Game 1 vs. Game 0

- ▶ **Game change:**

- ▶ Game 1 is almost identical to Game 0 .
- ▶ *Only difference:* the line marked "\*" is deleted.
- ▶ No backward consistency check

## Game 1 vs. Game 0

- ▶ **Game change:**

- ▶ Game 1 is almost identical to Game 0 .
- ▶ *Only difference:* the line marked "\*" is deleted.
- ▶ No backward consistency check

- ▶ **Effect of the change:**

- ▶ For the  $i$ -th signing query, the value  $u_{ti}$  is sampled uniformly at random from  $\mathcal{G}$ .

## Game 1 vs. Game 0

- ▶ **Game change:**

- ▶ Game 1 is almost identical to Game 0 .
- ▶ *Only difference:* the line marked "\*" is deleted.
- ▶ No backward consistency check

- ▶ **Effect of the change:**

- ▶ For the  $i$ -th signing query, the value  $u_{ti}$  is sampled uniformly at random from  $\mathcal{G}$ .
- ▶ A “bad event” occurs if the random oracle was already queried at  $(m, u_{ti})$ :
  - ▶ directly by the adversary, or
  - ▶ indirectly via a previous signing query.

## Bounding the bad event

- ▶ **Bounding the bad event:**

- ▶ By the union bound, for a fixed signing query:

$$\Pr[\text{bad}] \leq \frac{Q_s + Q_{ro} + 1}{q}.$$

- ▶ Applying the union bound over all  $Q_s$  signing queries:

$$\Pr[\text{bad in any query}] \leq \frac{Q_s (Q_s + Q_{ro} + 1)}{q}.$$

- ▶ **Difference Lemma:**

$$|\Pr[W_1] - \Pr[W_0]| \leq \frac{Q_s (Q_s + Q_{ro} + 1)}{q}.$$

## From Game 1 to the $r$ -Impersonation Game

- ▶ **Purpose of the change (Game 1):**
  - ▶ Each signing query uses a *fresh random challenge*.
  - ▶ This matches the behavior of an honest verifier in Schnorr's identification protocol.

## From Game 1 to the $r$ -Impersonation Game

- ▶ **Purpose of the change (Game 1):**

- ▶ Each signing query uses a *fresh random challenge*.
- ▶ This matches the behavior of an honest verifier in Schnorr's identification protocol.

- ▶ **Consequence:**

- ▶ Game 1 can be embedded into an  $r$ -impersonation eavesdropping attack.
- ▶ Set  $r = Q_{ro} + 1$  concurrent verifiers.

Obtain verification key  $u \leftarrow g^{\alpha'}$  from challenger.  
 Obtain eaves conv  $(u_{ti}, c_i, \alpha_{zi}) ; i = 1, \dots Q$  from challenger  
 Initialize:  $RO : M \times G \rightarrow C$  ,  $Explicit : M \times G \rightarrow \mathbb{Z}$   
 Send the public key  $u$  to A

1) Upon receiving the i-th signing query  $m_i \in M$ :

$$RO[m_i, u_{ti}] \leftarrow c_i$$

\*

Send  $(u_{ti}, \alpha_{zi})$  to A

2) Upon receiving j-th RO query  $(\widehat{m}_j, \widehat{u}_j) \in M \times G$

if  $(\widehat{m}_j, \widehat{u}_j) \notin \text{Domain}(RO)$  then

initiate the impersonation attempt with verifier  $V_j$

send  $\widehat{u}_j$  to  $V_j$  who responds with challenge  $\widehat{c}_j$

$$RO[(\widehat{m}_j, \widehat{u}_j)] \leftarrow \widehat{c}_j, \text{Explicit}[(\widehat{m}_j, \widehat{u}_j)] \leftarrow j$$

send  $\text{Map}[(\widehat{m}_j, \widehat{u}_j)]$  to A

3) Upon receiving a forgery attempt  $(m, u_t, \alpha_z)$ :

send the final message  $\alpha_z$  to  $V_j$  ,  $j = \text{Explicit}[m, u_t]$



Prepare set of signing queries:  
 $\{m_i \mid i = 1, \dots Q_s\}$



Prepare set of RO queries:  
 $(\widehat{m}_j, \widehat{u}_j)$

Adversary B Simulation game



- ▶ **Resulting equivalence:**

$$\Pr[W_1] = rID2Adv[\mathcal{B}, \mathcal{I}_{Sch}, r].$$

- ▶ Putting it all together we get,

$$SIG^{ro}adv[A, S_{Sch}] \leq Q_s (Q_s + Q_{ro} + 1)/q + rID2adv[B, I_{sch}, r]$$

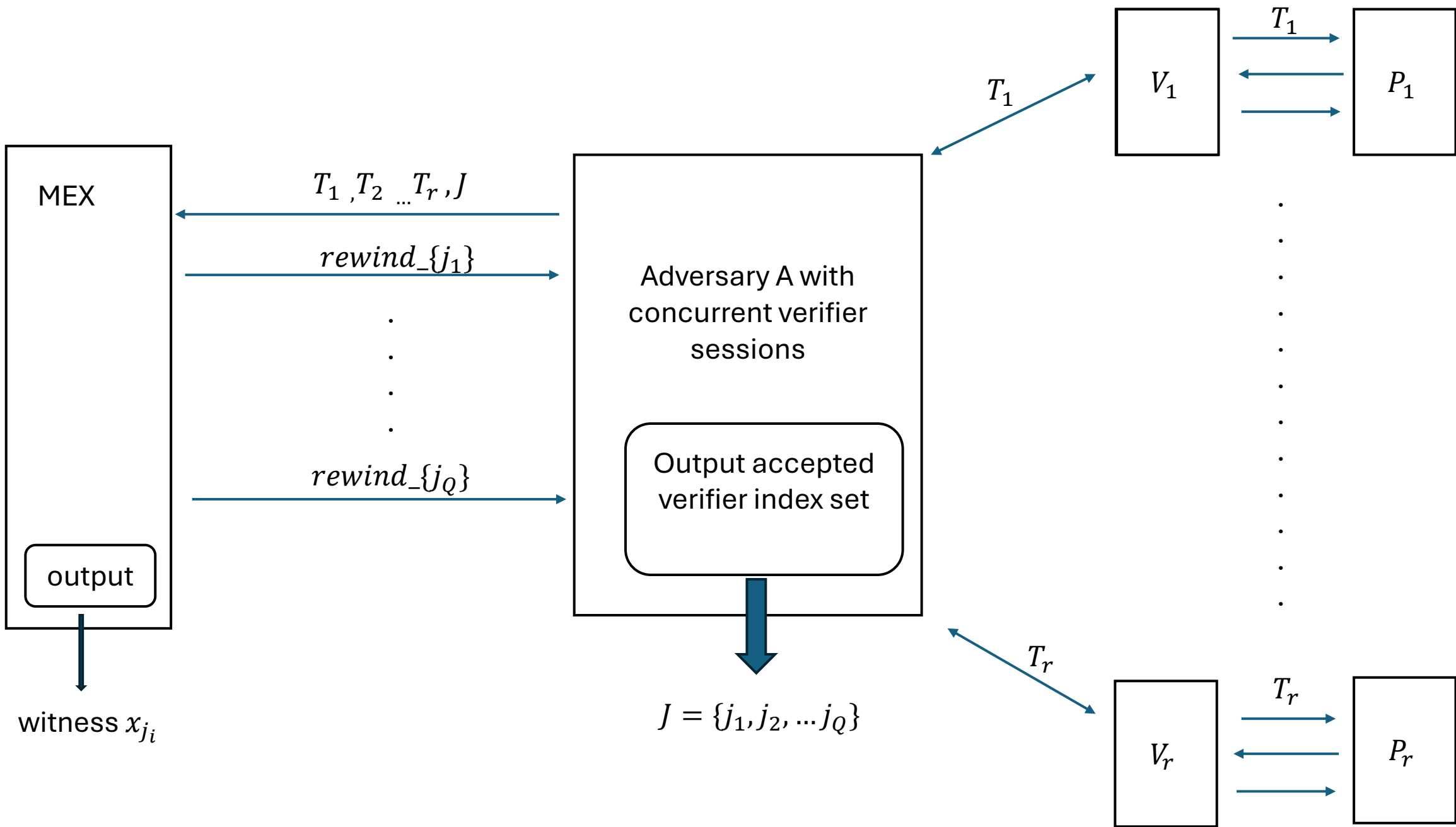
## Quick Recap

- ▶ We have now achieved non-interactive ID protocols using Fiat Shamir transformation.
- ▶ One key component for security for verifier is the "**rewinding technique**" for knowledge extraction.

**Can we extend above ideas to multiple concurrent ID-sessions ?**

## Multi-Extractor (MEX): Intuition

- ▶ MEX is a **black-box extraction tool** for Sigma protocols.
- ▶ It is used when an adversary produces **multiple accepting transcripts** across concurrent sessions.
- ▶ Goal:  
*Extract all corresponding witnesses from one successful adversarial run.*
- ▶ MEX relies only on:
  - ▶ special soundness of the protocol,
  - ▶ the ability to rewind the adversary in its **full**.



MEX Rewinding  
at index 2

Same as  $T_1$

·  
·

Rerun with  $ch^*$

·  
·

Same as  $T_r$

·  
·  
·  
·

A's random coin

$Com_2$

A's random coin

$ch_2$

A's random coin

$resp_2$

·  
·  
·  
·



$J' = \{j'_1, j'_2, \dots\}$

# Multi-Extractor (MEX): Output and Guarantees

- ▶ **Output:**
  - ▶ either a set of witnesses (one per accepted session), or
  - ▶ fail.
- ▶ If MEX outputs witnesses, they are **all valid**.
- ▶ MEX may fail with small probability, which can be made arbitrarily low.
- ▶ Running time and number of rewinds remain **polynomial**.

Lemma:

**Sigma Protocols are Multi-extractible.**

# Encrypt-Then-Prove Paradigm

- ▶ Goal: protect ElGamal encryption against **chosen-ciphertext attacks**.



# Encrypt-Then-Prove Paradigm

- ▶ Goal: protect ElGamal encryption against **chosen-ciphertext attacks**.
- ▶ Idea:
  - ▶ Encrypt as usual.
  - ▶ Attach a **non-interactive proof of knowledge** that the ciphertext is well-formed.

# Encrypt-Then-Prove Paradigm

- ▶ Goal: protect ElGamal encryption against **chosen-ciphertext attacks**.
- ▶ Idea:
  - ▶ Encrypt as usual.
  - ▶ Attach a **non-interactive proof of knowledge** that the ciphertext is well-formed.
- ▶ Decryption rule:
  - ▶ Verify the proof first.
  - ▶ Decrypt *only if* the proof is valid.

## Base Scheme: Multiplicative ElGamal

- ▶ Public parameters:
  - ▶ Cyclic group  $G = \langle g \rangle$  of prime order  $q$ .
- ▶ Keys:
  - ▶ Secret key:  $\alpha \in \mathbb{Z}_q$ .
  - ▶ Public key:  $u = g^\alpha$ .

## Base Scheme: Multiplicative ElGamal

- ▶ Public parameters:
  - ▶ Cyclic group  $G = \langle g \rangle$  of prime order  $q$ .
- ▶ Keys:
  - ▶ Secret key:  $\alpha \in \mathbb{Z}_q$ .
  - ▶ Public key:  $u = g^\alpha$ .
- ▶ Encryption of  $m \in G$ :

$$\beta \leftarrow \mathbb{Z}_q, \quad v = g^\beta, \quad e = u^\beta \cdot m$$

*send*  $(v, e)$

## Base Scheme: Multiplicative ElGamal

- ▶ Public parameters:
  - ▶ Cyclic group  $G = \langle g \rangle$  of prime order  $q$ .
- ▶ Keys:
  - ▶ Secret key:  $\alpha \in \mathbb{Z}_q$ .
  - ▶ Public key:  $u = g^\alpha$ .
- ▶ Encryption of  $m \in G$ :

$$\beta \leftarrow \mathbb{Z}_q, \quad v = g^\beta, \quad e = u^\beta \cdot m$$

*send*  $(v, e)$

- ▶ Decryption:

$$m = e/v^\alpha.$$

- ▶ Secure under DDH, but **insecure against CCA**.

## Adding a Proof of Knowledge

- ▶ Augment ciphertext:

$$(v, e) \longrightarrow (v, e, \pi).$$

- ▶ Proof  $\pi$ :

- ▶ Proves knowledge of  $\beta$  such that  $v = g^\beta$ .

- ▶ Intuition:

- ▶ Knowing  $\beta$  implies knowing  $m = e/u^\beta$ .

- ▶ Decryption:

- ▶ Reject malformed ciphertexts before decryption.

# Underlying Sigma Protocol

- ▶ Relation:

$$\mathcal{R} = \{(\beta, (u, v, e)) : g^\beta = v\}.$$

- ▶ Use Schnorr's  $\Sigma$ -protocol:

- ▶ Commitment:  $v_t = g^{\beta_t}$ .
- ▶ Challenge:  $c \in \mathcal{C} \subseteq \mathbb{Z}_q$ .
- ▶ Response:  $\beta_z = \beta_t + \beta c$ .

# Underlying Sigma Protocol

- ▶ Relation:

$$\mathcal{R} = \{(\beta, (u, v, e)) : g^\beta = v\}.$$

- ▶ Use Schnorr's  $\Sigma$ -protocol:

- ▶ Commitment:  $v_t = g^{\beta_t}$ .
- ▶ Challenge:  $c \in \mathcal{C} \subseteq \mathbb{Z}_q$ .
- ▶ Response:  $\beta_z = \beta_t + \beta c$ .

- ▶ Make the proof non-interactive using Fiat–Shamir.



# Underlying Sigma Protocol

- ▶ Relation:

$$\mathcal{R} = \{(\beta, (u, v, e)) : g^\beta = v\}.$$

- ▶ Use Schnorr's  $\Sigma$ -protocol:

- ▶ Commitment:  $v_t = g^{\beta_t}$ .
- ▶ Challenge:  $c \in \mathcal{C} \subseteq \mathbb{Z}_q$ .
- ▶ Response:  $\beta_z = \beta_t + \beta c$ .

- ▶ Make the proof non-interactive using Fiat–Shamir.

- ▶ Challenge computation:

$$c = H((u, v, e), v_t).$$

- ▶ Ciphertext structure:

$$(v, e, \pi) = (v, e, (v_t, \beta_z)).$$

## Augmented ElGamal: $\mathcal{E}_{\text{aMEG}}$

- ▶ Encryption:
  - ▶ Standard ElGamal encryption.
  - ▶ Compute FS proof of knowledge of  $\beta$ .
- ▶ Decryption:
  - ▶ Recompute  $c = H((u, v, e), v_t)$ .
  - ▶ Verify  $g^{\beta_z} = v_t \cdot v^c$ .
  - ▶ Decrypt only if verification succeeds.

## Parallel 1CCA: Adversary Model

- ▶ The adversary is allowed **one** decryption query.
- ▶ The query is a **batch** of ciphertexts:

$$(v_1^*, e_1^*, \pi_1^*), \dots, (v_n^*, e_n^*, \pi_n^*)$$

- ▶ The batch query may be made:
  - ▶ before the challenge, or
  - ▶ after the challenge.
  - ▶ W.L.O.G: queries made after the challenge.

## Security Claim

### **Theorem (Parallel 1CCA Security):**

- ▶ If DDH assumption holds, then the encryption scheme  $\mathcal{E}_{\text{AMEG}}$  is parallel 1CCA secure in the random oracle model:

$$\alpha, g^\alpha \leftarrow G()$$

$$\beta \leftarrow \mathbb{Z}_q, v \leftarrow g^\beta, w \leftarrow u^\beta, e \leftarrow w.m$$

$$\beta_t \leftarrow \mathbb{Z}_q, v_t \leftarrow g^{\beta_t},$$

$$c \leftarrow H(u, v, e, v_t),$$

$$\beta_z \leftarrow \beta_t + \beta.c$$

$$\text{Ciphertext} = (v, e, \pi = (v_t, \beta_z))$$

Decryption :

$$c \leftarrow H(u, v, e, v_t)$$

$$\text{If } g^{\beta_z} = v_t . v^c$$

$$\text{then } w \leftarrow v^\alpha, m \leftarrow \frac{e}{w}, \text{ output } m$$

*else output reject*

$$u = g^\alpha$$

$$(v, e, \pi = (v_t, \beta_z))$$

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Generate batch decryption queries with valid proofs.

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Parallel 1-CCA game of  $\varepsilon_{mEG}$

Basic Elgamal  
challenger

$$u = g^\alpha$$

$$(v, e)$$

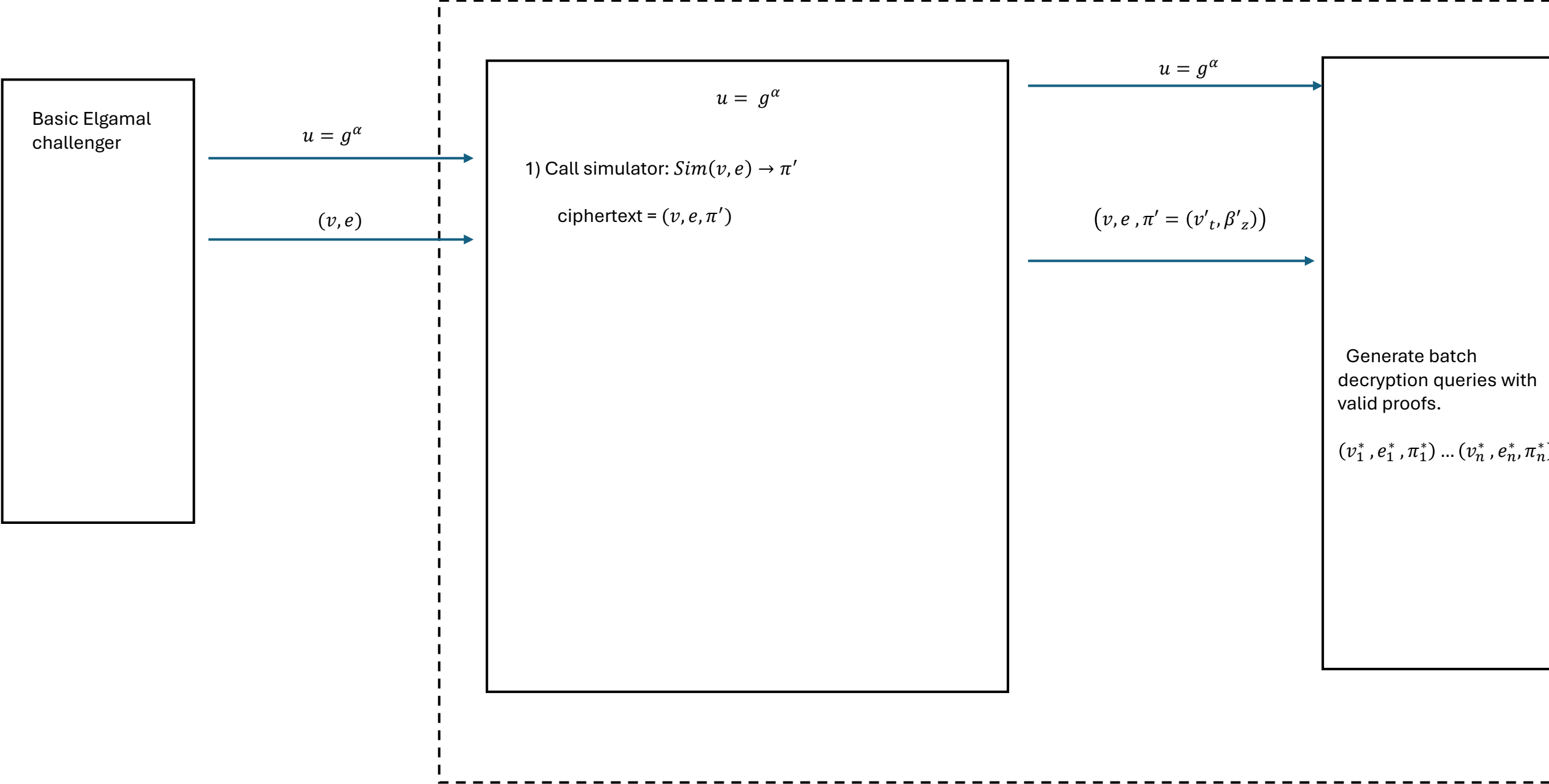
$$u = g^\alpha$$

$$u = g^\alpha$$

Generate batch  
decryption queries with  
valid proofs.

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Parallel 1-CCA security of  $\varepsilon_{mEG}$



Parallel 1-CCA security of  $\varepsilon_{mEG}$

Basic Elgamal  
challenger

$$u = g^\alpha$$

$$(v, e)$$

$$u = g^\alpha$$

1) Call simulator:  $\text{Sim}(v, e) \rightarrow \pi'$

ciphertext =  $(v, e, \pi')$

2) Define RO and handle the consistency.

$$u = g^\alpha$$

$$(v, e, \pi' = (v'_t, \beta'_z))$$

Generate batch  
decryption queries with  
valid proofs.

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Parallel 1-CCA security of  $\varepsilon_{mEG}$



Basic Elgamal  
challenger

$$u = g^\alpha$$

$$(v, e)$$

$$u = g^\alpha$$

1) Call simulator:  $\text{Sim}(v, e) \rightarrow \pi'$

ciphertext =  $(v, e, \pi')$

2) Define RO for consistent answers

3) Upon receiving batch decryption queries

$$u = g^\alpha$$

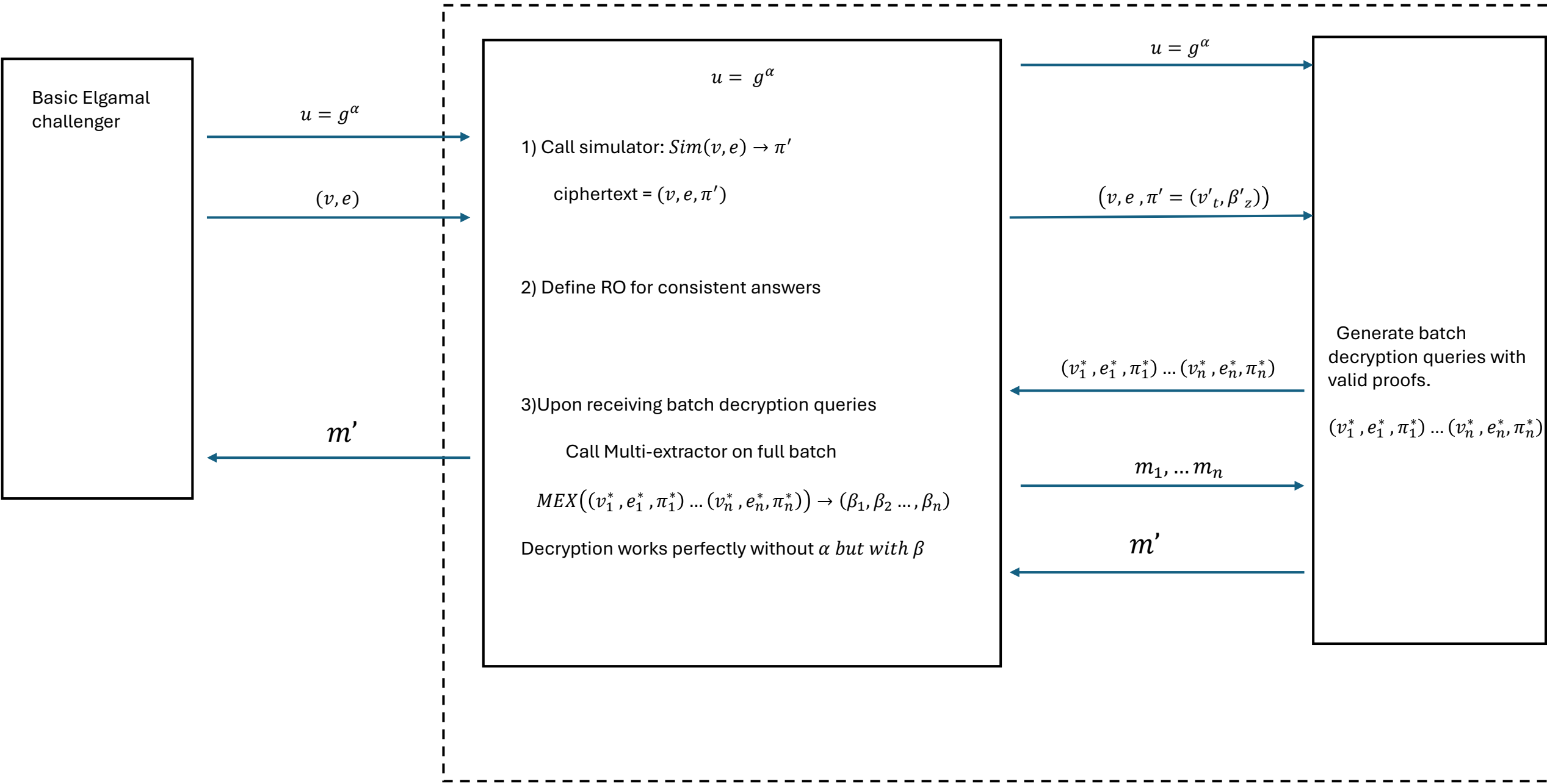
$$(v, e, \pi' = (v'_t, \beta'_z))$$

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Generate batch  
decryption queries with  
valid proofs.

$$(v_1^*, e_1^*, \pi_1^*) \dots (v_n^*, e_n^*, \pi_n^*)$$

Parallel 1-CCA security of  $\varepsilon_{mEG}$



Parallel 1-CCA security of  $\varepsilon_{mEG}$

## Adaptive security

**Can't we do adaptive CCA security in the same way ?**

## Adversary Strategy: Reverse Dependencies

- ▶ Adversary prepares ciphertexts  $(C_1, \dots, C_n)$ .
- ▶ Construction is done **backwards**:
  - ▶  $C_n$  is independent.
  - ▶  $C_{n-1}$  depends on  $c_n$ .
  - ▶  $C_{n-2}$  depends on  $(c_{n-1}, c_n)$ .

- ▶ In general:

$$C_k = \text{Enc}(m_k(c_{k+1}, \dots, c_n), \beta_k(c_{k+1}, \dots, c_n)).$$

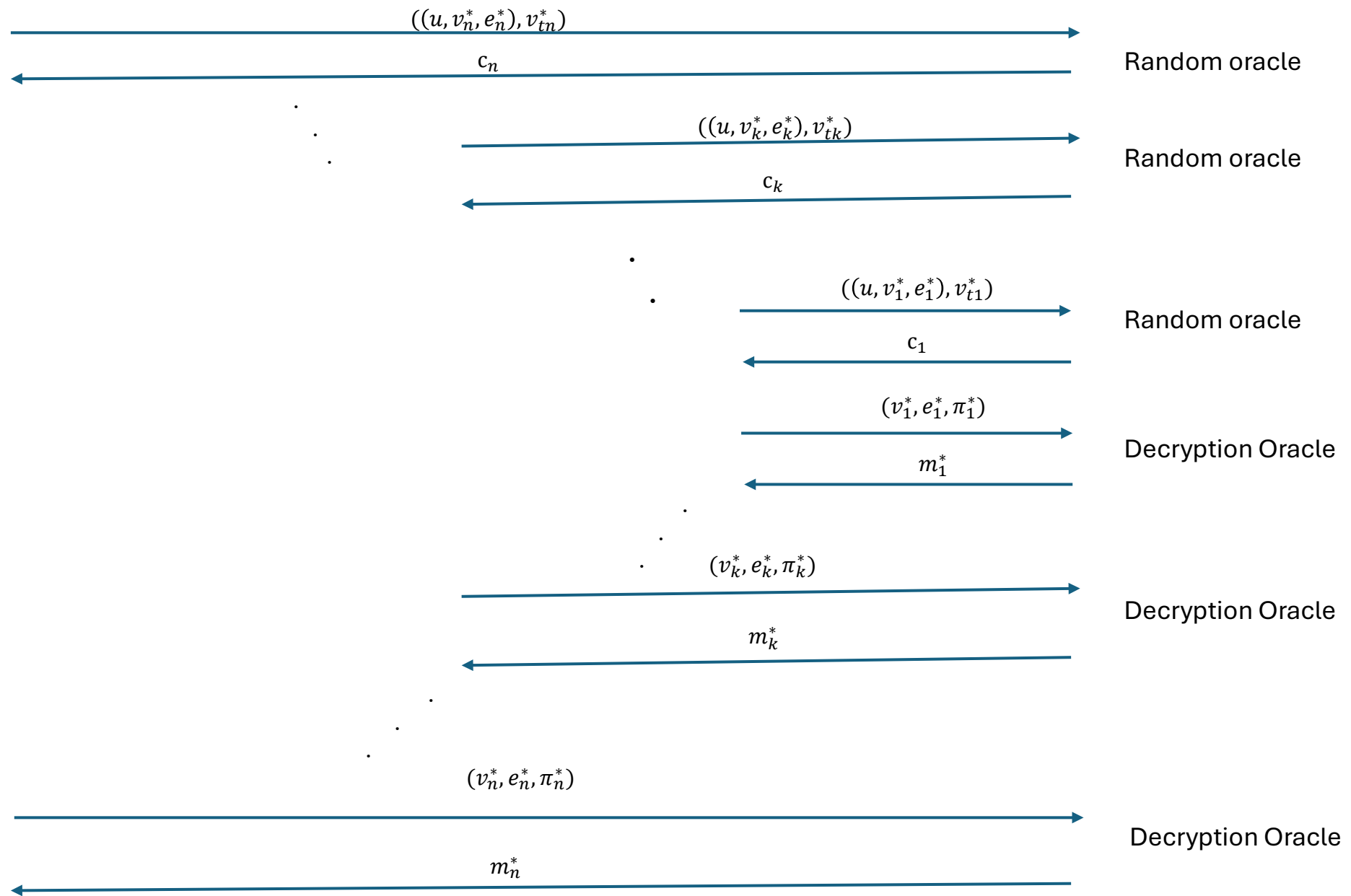
## Adversary Strategy: Reverse Dependencies

- ▶ Adversary prepares ciphertexts  $(C_1, \dots, C_n)$ .
- ▶ Construction is done **backwards**:
  - ▶  $C_n$  is independent.
  - ▶  $C_{n-1}$  depends on  $c_n$ .
  - ▶  $C_{n-2}$  depends on  $(c_{n-1}, c_n)$ .

- ▶ In general:

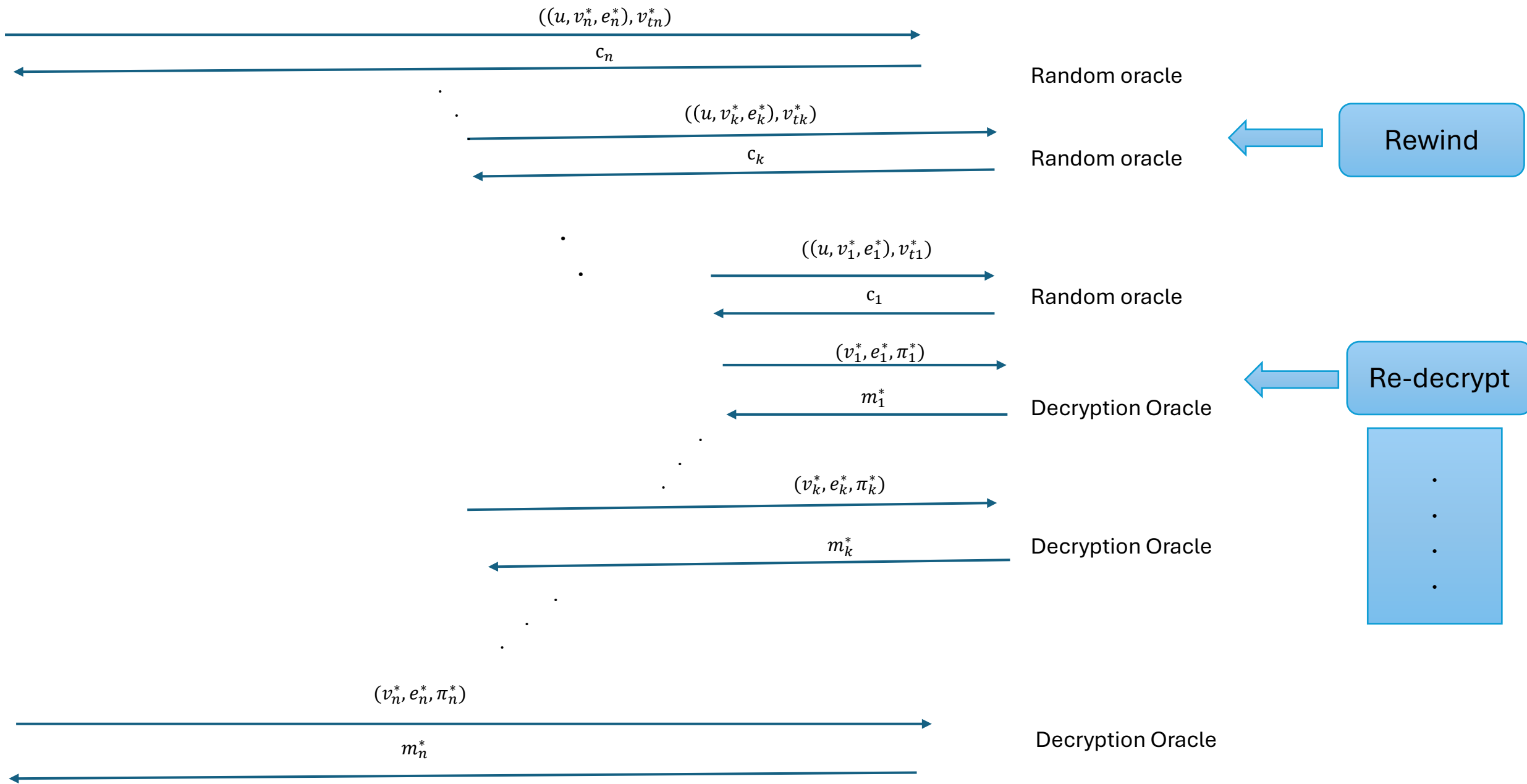
$$C_k = \text{Enc}(m_k(c_{k+1}, \dots, c_n), \beta_k(c_{k+1}, \dots, c_n)).$$

- ▶ Key property:
  - ▶ Changing any future challenge changes earlier ciphertexts.



## Why Rewinding Breaks under Adaptive CCA

- ▶ To extract for  $C_k$ , extractor rewinds to change  $c_k$ .
- ▶ But rewind point is *before* ciphertexts  $C_1, \dots, C_{k-1}$  were formed.
- ▶ In the re-run:
  - ▶ Future challenges change.
  - ▶ Earlier ciphertexts are recomputed.
- ▶ Result:
  - ▶ The extractor encounters a *new* first ciphertext.
  - ▶ Previously extracted witnesses are no longer useful.





## Recursive Extraction Problem

- ▶ Rewinding to extract for  $C_k$  creates a new execution branch.
- ▶ In that branch:
  - ▶ Must re-extract witnesses for  $C_1, \dots, C_{k-1}$ .
  - ▶ Each re-extraction may require further rewinds.
- ▶ Extraction becomes **nested and recursive**.

## Quantifying the Blow-Up

- ▶ Let  $f(k)$  = number of primitive extractions to answer  $k$  adaptive queries.
- ▶ Recurrence:

$$f(k) = 2f(k-1) + 1, \quad f(0) = 0.$$

- ▶ Solution:

$$f(k) = 2^k - 1.$$

- ▶ Exponential blow up for multi-extractor.

## Limitations of FS-transform

- ▶ Not much tighter security bounds.
- ▶ Rewinding may blow up extractors.

## Limitations of FS-transform

- ▶ Not much tighter security bounds.
- ▶ Rewinding may blow up extractors.

**Need better solution!**

Q. Can we obtain 2 transcripts for extraction without rewinding?

Q. Can we obtain 2 transcripts for extraction without rewinding?

- ▶ Positive answer by **Mark Fischlin** in 2005 by technique defined as **Straight line extraction / Online extractor**
- ▶ idea is that the moment query came for decryption right-away extract the witness out of it.

# Fiat–Shamir Proof of Knowledge: Syntax

- ▶ Relation:  $W = (W_k)_k$ , with instances  $x$  and witnesses  $w$ .
- ▶ Challenge length:  $k$  bits.
- ▶ Algorithms:

$$P_{\text{FS}} = (P_0, P_1), \quad V_{\text{FS}} = (V_0, V_1).$$

- ▶ Protocol flow:
  - ▶ Commitment:  $\text{com} \leftarrow P_0(x, w)$
  - ▶ Challenge:  $\text{ch} \leftarrow V_0(x, \text{com}) \in \{0, 1\}^k$
  - ▶ Response:  $\text{resp} \leftarrow P_1(x, w, \text{com}, \text{ch})$
  - ▶ Verify:  $V_1(x, \text{com}, \text{ch}, \text{resp}) \in \{0, 1\}$

# Execution Notation

- ▶ Interactive execution is denoted by:

$$(\text{com}, \text{ch}, \text{resp}) \leftarrow P_{\text{FS}}(x, w; V_0(x, \cdot)).$$

- ▶ Randomness is over:
  - ▶ prover randomness,
  - ▶ verifier (challenge) randomness.



# Completeness and Public-Coin Property

- ▶ **Completeness:**

- ▶ If  $(x, w) \in W_k$  and both parties are honest,
- ▶ the verifier always accepts.

$$\Pr [V_1(x, \text{com}, \text{ch}, \text{resp}) = 1] = 1.$$

- ▶ **Public-coin:**

- ▶ Challenge  $\text{ch}$  is uniform in  $\{0, 1\}^k$ ,
- ▶ independent of the witness.

# Commitment Entropy and Uniqueness

- ▶ **Commitment entropy:**
  - ▶  $\text{com} \leftarrow P_0(x, w)$  has superlogarithmic min-entropy.
  - ▶ Prevents adversarial guessing or fixing challenges.
- ▶ **Unique responses (quasi-uniqueness):**
  - ▶ For fixed  $(x, \text{com}, \text{ch})$ ,
  - ▶ there is (essentially) a unique accepting response.

## Special Soundness (Extraction)

- ▶ From two accepting transcripts:

$$(\text{com}, \text{ch}, \text{resp}), \quad (\text{com}, \text{ch}', \text{resp}') \quad \text{with } \text{ch} \neq \text{ch}',$$

- ▶ There exists a PPT extractor  $K_{\text{FS}}$  such that:

$$w \leftarrow K_{\text{FS}}(\cdot) \Rightarrow (x, w) \in W_k.$$

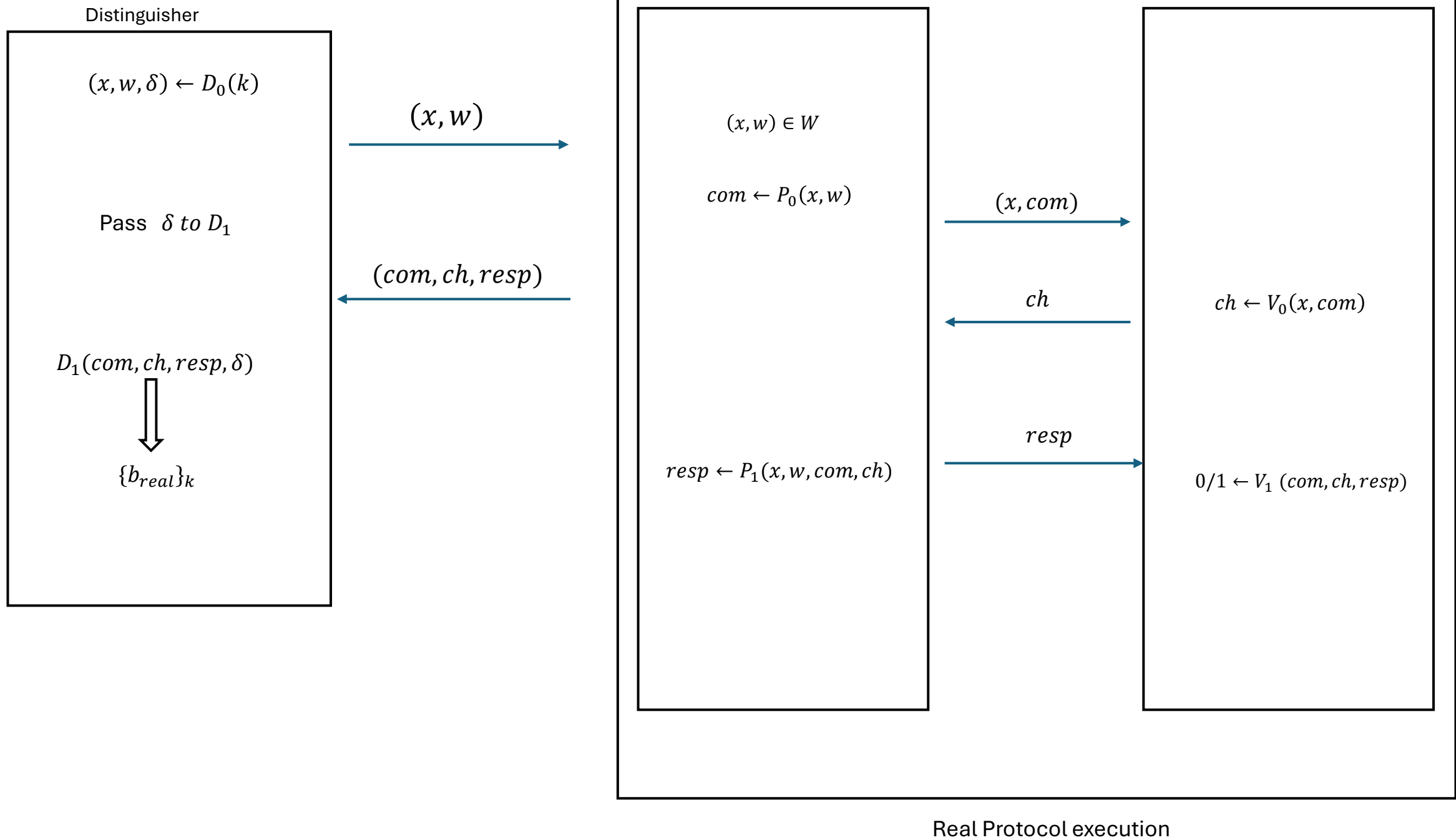
- ▶ This is the **proof of knowledge** guarantee.

# Honest-Verifier Zero Knowledge (HVZK)

- ▶ There exists a PPT simulator  $Z_{FS}$ .
- ▶ Simulator outputs:  
 $(com, ch, resp) \approx \text{real transcript.}$
- ▶ Indistinguishable from a real interaction with an honest verifier.

# HVZK: Real vs Simulated Experiments

- ▶ **Real world:**
  - ▶ Honest prover interacts with honest verifier.
- ▶ **Simulated world:**
  - ▶ Simulator generates transcript without witness.
- ▶ No PPT distinguisher can tell the difference.



Distinguisher

$(x, w, \delta) \leftarrow D_0(k)$

Pass  $\delta$  to  $D_1$

$D_1(com, ch, resp, \delta)$



$\{b_{sim}\}_k$

$(x, \cdot)$



$(com, ch, resp)$

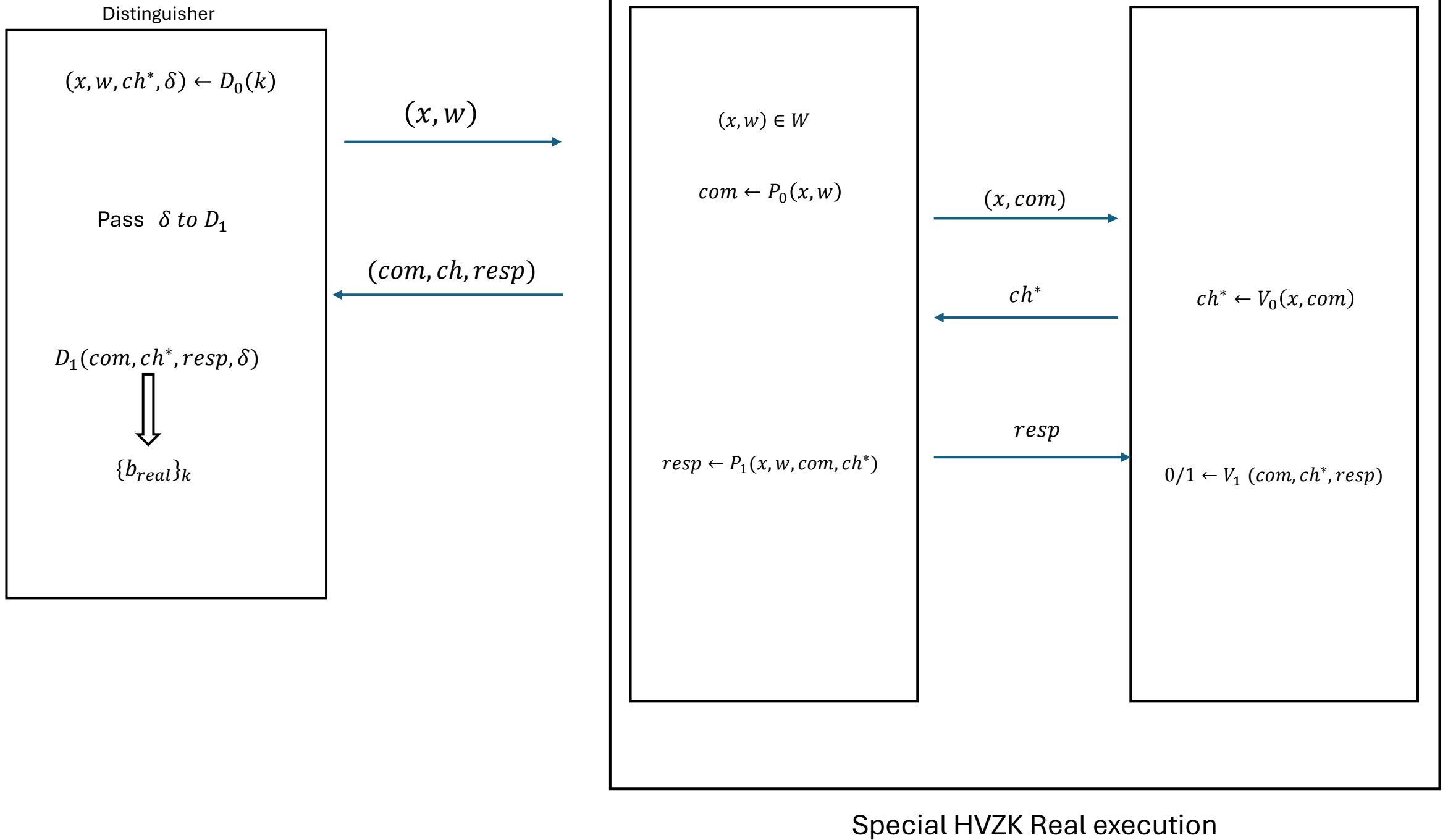


$com \leftarrow Z_0(x, yes)$

$ch \leftarrow \mathcal{C}$

$resp \leftarrow Z_1(x, com, ch)$

Simulation Protocol





Distinguisher

$(x, w, ch^*, \delta) \leftarrow D_0(k)$

Pass  $\delta$  to  $D_1$

$D_1(com, ch^*, resp, \delta)$



$\{b_{sim}\}_k$

$(x, ch^*)$

$(com, ch^*, resp)$

$com \leftarrow Z_0(x, ch^*, yes)$

$resp \leftarrow Z_1(x, com, ch^*)$

Simulation in special HVZK

**Non-interactive Zero Knowledge with online extractor.**

# Setting and Syntax

- ▶ Relation:  $W = (W_k)_k$  with statements  $x$  and witnesses  $w$ .
- ▶ Model: Random Oracle Model (ROM) with oracle  $H$ .
- ▶ Algorithms:
  - ▶ Prover:  $P^H(x, w) \rightarrow \pi$
  - ▶ Verifier:  $V^H(x, \pi) \in \{0, 1\}$
- ▶ Interaction is *non-interactive*:

$$(x, w) \xrightarrow{P^H} \pi \xrightarrow{V^H} \text{accept/reject}$$

# Completeness

- ▶ Honest prover with a valid witness always succeeds.
- ▶ Holds for every random oracle  $H$ .

$$\forall (x, w) \in W_k : \Pr [V^H(x, P^H(x, w)) = 1] = 1$$

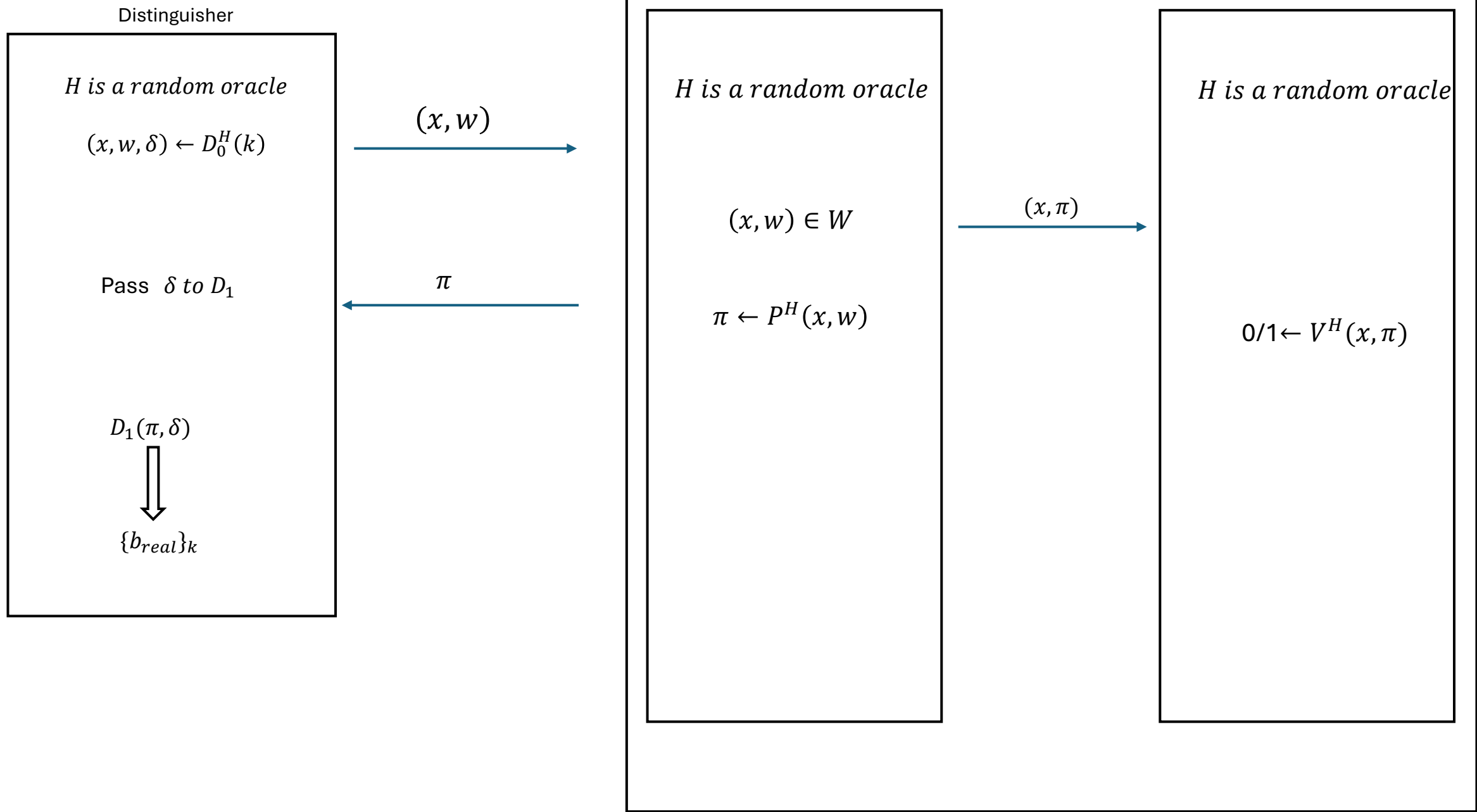
## Zero Knowledge in the ROM: Intuition

- ▶ Proofs reveal no information beyond validity.
- ▶ Even when the verifier controls oracle queries.
- ▶ Achieved via a *programmable random oracle*.

Real execution  $\approx$  Simulated execution

## Zero Knowledge: Real Experiment

- ▶ Oracle  $H$  is chosen uniformly at random.
- ▶ Distinguisher  $D = (D_0, D_1)$  runs with oracle access.
  1.  $(x, w, \sigma) \leftarrow D_0^H(k)$
  2. If  $(x, w) \in W_k$ :  $\pi \leftarrow P^H(x, w)$
  3. Else:  $\pi \leftarrow \pi_{\text{invalid}}$
  4. Output  $D_1^H(\pi, \sigma)$



## Zero Knowledge: Simulated Experiment

► Simulator  $Z = (Z_0, Z_1)$  controls the oracle.

1.  $Z_0$  outputs initial oracle  $H_0$  and state  $\tau$

2.  $(x, w, \sigma) \leftarrow D_0^{H_0}(k)$

3.  $Z_1(k, x, \tau, b)$  outputs:

► completed oracle  $H_1 \supseteq H_0$

► proof  $\pi$

4. Output  $D_1^{H_1}(\pi, \sigma)$

►  $b = \text{yes}$  if  $(x, w) \in W_k$ , else no

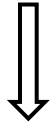


Distinguisher

$$(x, w, \delta) \leftarrow D_0^{H_0}(k)$$

Pass  $\delta$  to  $D_1$

$$D_1^{H_1}(\pi, \delta)$$



$\{b_{sim}\}_k$

$(x, \cdot)$

$\pi$

*$H$  is a random oracle*

$$(H_0, \tau) \leftarrow Z_0(k)$$

*pass the state  $\tau$  to  $Z_1$*

$$(H_1, \pi) \leftarrow Z_1(x, \tau, k, \text{yes})$$

Zero Knowledge in ROM- simulated execution

## Online Extractor: Formal Definition

- ▶ Let  $A$  be any PPT prover (possibly malicious).
- ▶  $A$  interacts with oracle  $H$  and outputs  $(x, \pi)$ .
- ▶ Let  $Q_H^A$  be the full list of oracle queries and answers.

$$w \leftarrow K(x, \pi, Q_H^A)$$

## Extractor Soundness Guarantee

- ▶ If verifier accepts, extractor recovers a valid witness.
- ▶ Failure probability is negligible.

$\Pr [V^H(x, \pi) = 1 \wedge (x, w) \notin W_k]$  is negligible

## **Fischlin Transform Generic Construction**

# Underlying Ingredients

- ▶ Relation:  $W = (W_k)_k$
- ▶ Base protocol:
  - ▶ Fiat–Shamir  $\Sigma$ -protocol  $(P_{\text{FS}}, V_{\text{FS}})$
  - ▶ Challenge length:

$$\ell_{\text{FS}} = \ell(k) = O(\log k)$$

- ▶ Random oracle:

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^b$$

## Concrete Parameters

All parameters depend on the security parameter  $k$ .

$b$  hash output length,  
 $t$  challenge search width,  
 $r$  number of repetitions,  
 $S$  acceptance threshold.

Constraints:

$$b, r, t = O(\log k), \quad b \leq t \leq \ell_{\text{FS}}$$
$$br = \omega(\log k), \quad 2^{t-b} = \omega(\log k), \quad S = O(r)$$

## Notation per Repetition

For each repetition  $i \in \{1, \dots, r\}$ :

- ▶ Commitment:  $\text{com}_i$
- ▶ Challenge:  $\text{ch}_i \in \{0, \dots, 2^t - 1\}$
- ▶ Response:  $\text{resp}_i$
- ▶ Hash score:

$$s_i := H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i)$$

$$\mathbf{com} = (\text{com}_1, \dots, \text{com}_r)$$

## Prover Algorithm: Commit Phase

1. Run  $P_{\text{FS}}(x, w)$  independently  $r$  times.
2. Stop after commitment phase.
3. Obtain:

$$\text{com}_1, \dots, \text{com}_r$$

4. Set:

$$\mathbf{com} = (\text{com}_1, \dots, \text{com}_r)$$



## Prover Algorithm: Challenge Search

For each repetition  $i$ :

- ▶ For all  $\text{ch} \in \{0, \dots, 2^t - 1\}$ :
  - ▶ Compute  $\text{resp}(\text{ch})$  using  $\text{com}_i$
  - ▶ Compute:

$$h := H(x, \mathbf{com}, i, \text{ch}, \text{resp}(\text{ch}))$$

## Prover Algorithm: Selection Rule

For repetition  $i$ :

- ▶ If  $\exists \text{ch}$  with  $h = 0$ :

$$\text{ch}_i := \text{ch}, \quad \text{resp}_i := \text{resp}(\text{ch})$$

- ▶ Otherwise:

$$\text{ch}_i := \arg \min_{\text{ch}} h$$

- ▶ Set  $s_i := h$  accordingly.

## Final Proof Format

The non-interactive proof is:

$$\pi = ((\text{com}_i, \text{ch}_i, \text{resp}_i))_{i=1}^r$$

- ▶ Verifier sees exactly one transcript per repetition.
- ▶ Prover internally explores many challenges.

# Verifier Algorithm

1. Check FS validity for all  $i$ :

$$V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1$$

2. Compute:

$$s_i := H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i)$$

3. Compute sum:

$$T := \sum_{i=1}^r s_i$$

4. Accept if  $T \leq S$ .

## Final Acceptance Condition

Verifier accepts iff:

$$\forall i : V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1$$

and

$$\sum_{i=1}^r H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i) \leq S$$

## Prover

1. Run  $P_{FS}(x, w)$  independently  $r$  times
2. stop after commitment phase
3. Obtain  $\overline{com} = (com_1, \dots, com_r)$
4. For each repetition  $i = 1, \dots, r$  :
  - For all  $ch \in \{0, \dots, 2^t - 1\}$ 
    - Compute  $resp(ch)$  using  $com_i$
    - Compute  $h = H(x, \overline{com}, i, ch, resp(ch))$
5. For repetition  $i$ :
  - if  $\exists ch$  with  $h = 0$
  - then  $ch_i = ch, resp_i = resp(ch)$
  - else
  - choose the one with mini hash score.

Final set:  $\pi = (com_i, ch_i, resp_i)_{i=1}^r$

## Verifier

1. Check FS validity for all  $i$ :

$$V_{1,FS}(x, com_i, ch_i, resp_i) = 1$$

2. Compute:

$$s_i = H(x, \overline{com}, i, ch_i, resp_i)$$

3. Compute Sum:

$$T = \sum s_i$$

Accept if  $T \leq S$

$\pi$

## Fischlin Transformation

# Completeness: Statement

## Definition (Completeness)

For all security parameters  $k$ , all  $(x, w) \in W_k$ , and all random oracles  $H$ ,

$$\Pr [V^H(x, P^H(x, w)) = 1] = 1 - \text{negl}(k).$$

- ▶ Honest prover knows a witness  $w$ .
- ▶ FS verification always succeeds.
- ▶ Only possible failure: the hash-sum test.

## What Can Cause Rejection?

Verifier accepts iff:

$$T := \sum_{i=1}^r s_i \leq S,$$

where

$$s_i = H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i) \in \{0, \dots, 2^b - 1\}.$$

- ▶ FS checks pass with probability 1.
- ▶ Completeness reduces to bounding:

$$\Pr[T > S].$$



## Step 1: Per-Repetition Tail Bound

For a single hash value  $X$ :

$$\Pr[X \leq S] = \frac{S+1}{2^b}.$$

Probability that *none* of the  $2^t$  trials is  $\leq S$ :

$$\left(1 - \frac{S+1}{2^b}\right)^{2^t} \leq e^{-(S+1)2^{t-b}}.$$

► Hence:

$$\Pr[s_i > S] \leq e^{-(S+1)2^{t-b}}.$$

## Step 2: Any Large Hash is Unlikely

By union bound over  $r$  repetitions:

$$\Pr[\exists i : s_i > S] \leq r \cdot e^{-(S+1)2^{t-b}}.$$

- ▶  $r = O(\log k)$
- ▶  $2^{t-b} = \omega(\log k)$

$$\Rightarrow \Pr[\exists i : s_i > S] = \text{negl}(k).$$

## Remaining Bad Event

With overwhelming probability:

$$\forall i : s_i \leq S.$$

But verifier still rejects if:

$$T = \sum_{i=1}^r s_i > S.$$

- ▶ Now  $S + 1 \leq T \leq rS$ .
- ▶ We must bound  $\Pr[S < T \leq rS]$ .

## Step 3: Counting Bad Sums

Fix  $T$  with  $S < T \leq rS$ .

- ▶ Number of nonnegative integer partition:

$$(p_1, \dots, p_r) \text{ with } \sum p_i = T$$

- ▶ Number of partitions:

$$\binom{T + r - 1}{r - 1}.$$

Upper bound:

$$\binom{T + r - 1}{r - 1} \leq \left( \frac{e(rS + r - 1)}{r - 1} \right)^{r-1} \leq (e(2S + 1))^{r-1}.$$

## Step 4: Probability of One Fixed Partition

Fix  $(p_1, \dots, p_r)$  with  $\sum p_i = T$ .

For repetition  $i$ :

$$\prod_{i=1}^r \Pr[s_i \geq p_i] \leq \prod_{i=1}^r \left(1 - \frac{p_i}{2^b}\right)^{2^t} \leq \prod_{i=1}^r e^{-p_i 2^{t-b}} \leq e^{-\sum (p_i) 2^{t-b}} \leq e^{-T \cdot 2^{t-b}} \leq e^{-(S+1) \cdot 2^{t-b}}.$$

Independence gives:

$$\Pr[(s_1, \dots, s_r)] \leq e^{-T 2^{t-b}}.$$

## Step 5: Final Probability Bound

For fixed  $T$ :

$$\Pr[\sum s_i = T] \leq (e(2S + 1))^{r-1} \cdot e^{-T2^{t-b}}.$$

Summing over  $T = S + 1, \dots, rS$ :

$$\Pr[S < T \leq rS] \leq \exp(r \ln(e(2S + 1))) - (S + 1)2^{t-b}.$$

# Negligibility and Completeness

- ▶  $r \ln(e(2S + 1)) = O(\log k)$
- ▶  $(S + 1)2^{t-b} = \omega(\log k)$

$$\Rightarrow \Pr[T > S] = \text{negl}(k).$$

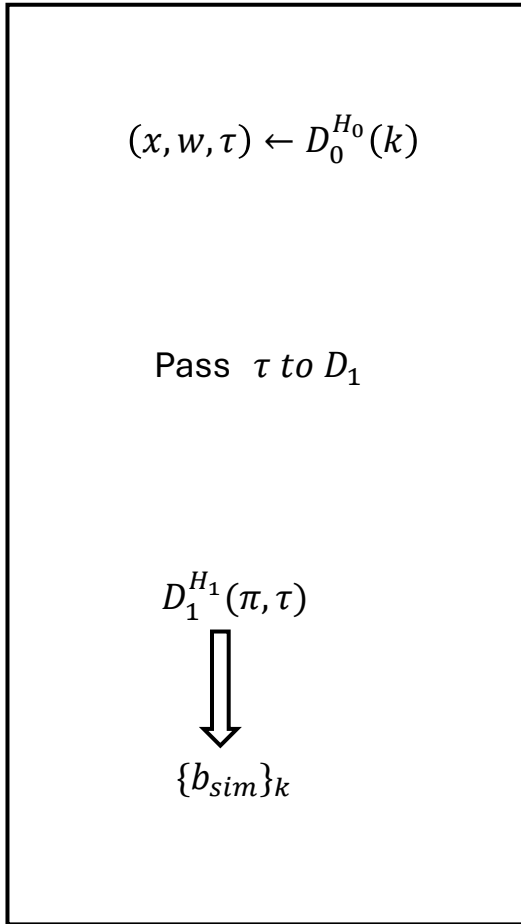
## Conclusion

An honest prover is accepted with overwhelming probability.

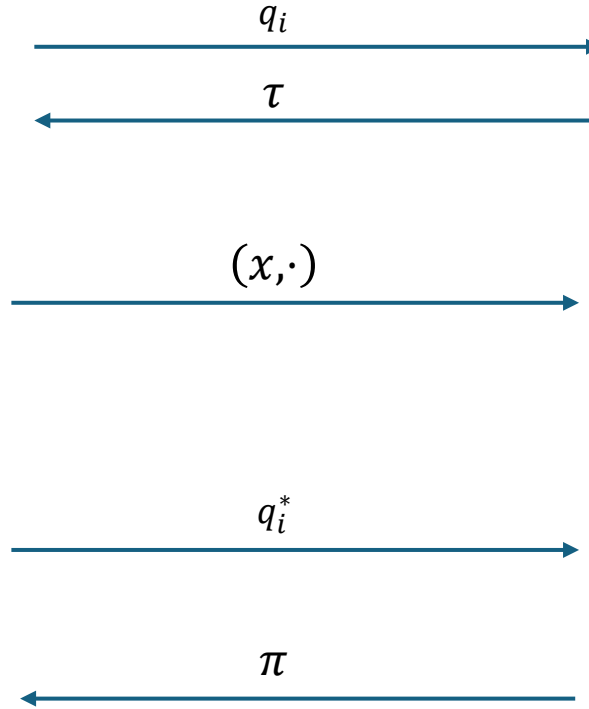
# Zero Knowledge

We will see the simulator for this.





Zero Knowledge Simulator



### $Z_0(k)$ setting oracle first stage( $\tau$ )

- i) if  $q \in H_0$  return  $H_0[q]$
- ii) else  $y \leftarrow \{0,1\}^b$ , set  $H_0[q] = y$ .
- 3) Return to  $D_0$  the simulated first stage view with  $H_0$

### $Z_1(x, k, \tau, \text{yes})$ simulating the transcript

1. For  $i = 1$  to  $r$ :
  - (a) Sample independent  $2^t$  uniform  $b - \text{bit strings}$
  - (b) Define map using above sampling  $\tau_i: \{0,1\}^t \rightarrow \{0,1\}^b$
  - (c) Choose the  $ch_i$  with minimum  $\tau_i(ch_i)$
  - (d) Run  $Z_{FS}(x, ch_i)$  to obtain  $(com_i, ch_i, resp_i)$
2. Copy  $H_0$  to  $H_1$
3. For each  $i$ :
 

For the tuple  $q_i = (x, \overline{com}, i, ch_i, resp_i)$  define  $H_1(q_i) = \tau_i(ch_i)$   
 For  $q_i^* = (x, \overline{com}, i, ch_i^*, resp_i^*)$  s.t  $V_{FS}(q_i^*) = 1$ ;  $H_1(q_i^*) = \tau_i(ch_i^*)$
2. When  $D_1$  makes queries:
 

if  $q \in H_1$  return  $H_1[q]$ .  
 else sample  $y \leftarrow \{0,1\}^b$ , set  $H_1[q] = y$ , return  $y$ .
3. Output  $\pi = (com_i, ch_i, resp_i)$  for  $i = 1$  to  $r$

## Bad Event 1: InconsistentOracle (Idea)

- ▶ In the simulation, the random oracle is *programmed* at specific points.
- ▶ A bad event occurs if the distinguisher has already queried such a point and received a conflicting value.
- ▶ We show that this happens only with negligible probability.

## Bad Event 1: InconsistentOracle (Definition)

For each repetition  $i \in \{1, \dots, r\}$ , define the programmed oracle input

$$q_i := (x, \text{com}_i, i, \text{ch}_i, \text{resp}_i).$$

InconsistentOracle occurs if:

- ▶ during stage 1, the distinguisher  $\mathcal{D}_0$  queries  $q_i$ , and
- ▶ the oracle returns a value different from the simulator's choice  $\tau_i(\text{ch}_i)$ .

In this case, the simulator would be forced to overwrite a previous oracle answer.

## Bad Event 1: Probability Bound

- ▶ The distinguisher  $\mathcal{D}_0$  makes at most  $Q(k)$  oracle queries.
- ▶ Each commitment  $\text{com}_i$  is sampled with entropy  $\kappa(k)$ , which is exponentially large.
- ▶ Therefore, the probability that a single query equals  $q_i$  is at most

$$\frac{Q(k)}{2^{\kappa(k)}}.$$

- ▶ Taking a union bound over all  $r$  repetitions:

$$\Pr[\text{InconsistentOracle}] \leq \frac{r \cdot Q(k)}{2^{\kappa(k)}}.$$

Since  $\kappa(k)$  is superlogarithmic, this probability is negligible.

## Goal of Online Extraction

- ▶ We show that a witness can be extracted from any malicious prover *without rewinding*.
- ▶ Extraction is required only if the prover convinces the verifier of a false statement.
- ▶ Equivalently:

*If no witness exists, the verifier accepts only with negligible probability.*

## Key Extraction Idea

- ▶ The extractor observes all random oracle queries made by the prover.
- ▶ If it finds two accepting transcripts

$$(x, \text{com}, i, \text{ch}_i, \text{resp}_i) \quad \text{and} \quad (x, \text{com}, i, \text{ch}'_i, \text{resp}'_i),$$

with  $\text{ch}_i \neq \text{ch}'_i$ , then:

- ▶ The standard FS extractor  $K_{\text{FS}}$  recovers the witness.
- ▶ No rewinding is required — extraction is fully *online*.

## Extractor Setup

- ▶ Let  $A$  be a malicious prover with oracle access to  $H$ .
- ▶ Execution:
  - ▶  $H$  is a random oracle.
  - ▶  $(x, \pi) \leftarrow A^H(k)$ .
  - ▶  $Q_H(A)$ : list of all oracle queries made by  $A$ .
- ▶ The extractor receives:

$$(x, \pi, Q_H(A))$$

and must output a witness  $w$ .

## When Does Extraction Succeed?

The extractor succeeds if it finds, for some repetition index  $i$ , two oracle queries:

$$(x, \text{com}, i, \text{ch}_i, \text{resp}_i), \quad (x, \text{com}, i, \text{ch}'_i, \text{resp}'_i),$$

such that:

- ▶  $\text{ch}_i \neq \text{ch}'_i$ ,
- ▶ both are accepted by  $V_{\text{FS}}$ .

**Otherwise**, we must show that verifier acceptance is negligible.



## Fixing a Prefix

Fix:

- ▶ a statement  $x$ ,
- ▶ a commitment vector  $\widetilde{\text{com}}$ ,
- ▶ a repetition index  $i$ .

Assume the extractor does *not* find two accepting transcripts for this prefix.

We analyze the consequences of this assumption.

## Defining the Random Variables $s_i$

For each repetition  $i$ :

- ▶ If  $A$  makes a valid accepting oracle query, define

$$s_i := H(x, \widetilde{\text{com}}, i, \text{ch}_i, \text{resp}_i).$$

- ▶ Otherwise, define  $s_i$  as a fresh uniform  $b$ -bit value.

**Key observation:**

$s_1, \dots, s_r$  are independent and uniformly random.

## Verifier Acceptance Condition

The verifier accepts only if

$$T := s_1 + s_2 + \cdots + s_r \leq S.$$

Thus, to break extraction security, the adversary must:

- ▶ avoid producing two accepting transcripts, and
- ▶ still satisfy  $T \leq S$ .

We bound the probability of this event.

## Counting Favorable Sums

The number of ways to write

$$T = s_1 + \cdots + s_r \leq S$$

with  $s_i \geq 0$  is at most

$$\sum_{T=0}^S \binom{T+r-1}{r-1} \leq (S+1) \binom{S+r}{r-1} \leq (S+1) \left( \frac{e(S+r)}{r-1} \right)^{r-1}$$

Using  $\binom{n}{k} \leq (en/k)^k$ , this is polynomial in  $k$ .

## Probability Bound

- ▶ Each  $s_i$  is uniform in  $\{0, 1\}^b$ .
- ▶ Total possible tuples:  $2^{br}$ .
- ▶ since  $S = O(r)$
- ▶ Favorable tuples (with  $T \leq S$ ):  $\leq (S + 1) \left( \frac{e(S+r)}{r-1} \right)^{r-1} \leq (S + 1) \cdot 2^{\log(ec)r}$

Therefore,

$$\Pr[T \leq S] \leq \frac{\text{poly}(k)}{2^{br}},$$

which is negligible since  $br = \omega(\log k)$ .

## Union Bound Over All Prefixes

- ▶ The adversary makes at most  $Q$  oracle queries.
- ▶ Hence there are at most  $Q + 1$  relevant prefixes  $(x, \widetilde{\text{com}})$ .
- ▶ Apply a union bound over all prefixes.

Final bound:

$$\Pr[\text{Verifier accepts and extraction fails}] \leq (Q + 1)(S + 1) \cdot 2^{(\log(ec) - b)r},$$

which is negligible.

## Conclusion

- ▶ If the verifier accepts with non-negligible probability, extraction succeeds.
- ▶ Otherwise, acceptance probability is negligible.
- ▶ The extractor is:
  - ▶ online,
  - ▶ non-rewinding,
  - ▶ compatible with the random oracle model.

**This completes the proof of online knowledge extraction.**

## Simulation Soundness: Statement

### Theorem (Multiple Zero-Knowledge and Simulation Soundness)

Let  $(P_{\text{FS}}, V_{\text{FS}})$  be a Fiat–Shamir proof of knowledge for relation  $W$ , and let  $(P, V)$  be the derived NIZK protocol.

- ▶ **(Multiple Zero-Knowledge)** *There exists a PPT simulator  $Z$  such that for any PPT distinguisher  $D$ , the real and simulated experiments with multiple adaptively chosen theorems are computationally indistinguishable.*
- ▶ **(Simulation Soundness)** *There exists a PPT online extractor  $K$  such that for any PPT adversary  $B$  that receives simulated proofs, the probability that  $B$  outputs a valid proof  $(x, \pi)$  without a witness is negligible.*



## Multiple Zero-Knowledge: Setup

- ▶ Adversary  $D$  adaptively queries  $n$  statements  $x_1, \dots, x_n$ .
- ▶ In the *real experiment*,  $D$  interacts with:
  - ▶ A true random oracle  $H$
  - ▶ Honest prover  $P^H$  producing real proofs
- ▶ In the *simulated experiment*,  $D$  interacts with:
  - ▶ A simulator  $Z = (Z_0, \dots, Z_n)$
  - ▶ Incrementally programmed oracles  $H_0, \dots, H_n$

# Multiple Zero-Knowledge: Hybrid Argument

- ▶ Define hybrids  $H^{(0)}, \dots, H^{(n)}$ :
  - ▶  $H^{(0)}$ : all proofs are real
  - ▶  $H^{(i)}$ : first  $i$  proofs are simulated, rest are real
- ▶ Indistinguishability of  $H^{(i-1)}$  and  $H^{(i)}$  follows from:
  - ▶ Single-theorem zero-knowledge of FS
  - ▶ Consistent random oracle programming
- ▶ Apply a standard hybrid argument over  $i = 1, \dots, n$ .

# Bad Events and Oracle Consistency

## Bad Event 1: InconsistentOracle

- ▶  $D$  queries the oracle on a tuple later used by the simulator
- ▶ Probability bounded by commitment min-entropy

$$\Pr[\text{InconsistentOracle}] \leq \frac{n \cdot Q(k)}{2^{\kappa(k)}} \text{ (negligible)}$$

## Simulation Soundness: Goal and Setup

- ▶ Adversary  $B$  receives arbitrarily many simulated proofs
- ▶  $B$  outputs  $(x, \pi)$  such that  $V^{H_n}(x, \pi) = 1$
- ▶ Goal: extract witness  $w$  with  $(x, w) \in W$

### Extractor:

- ▶ Use the same *online extractor*  $K$  as in the single-theorem case
- ▶ Input:  $(x, \pi, Q_H^B)$

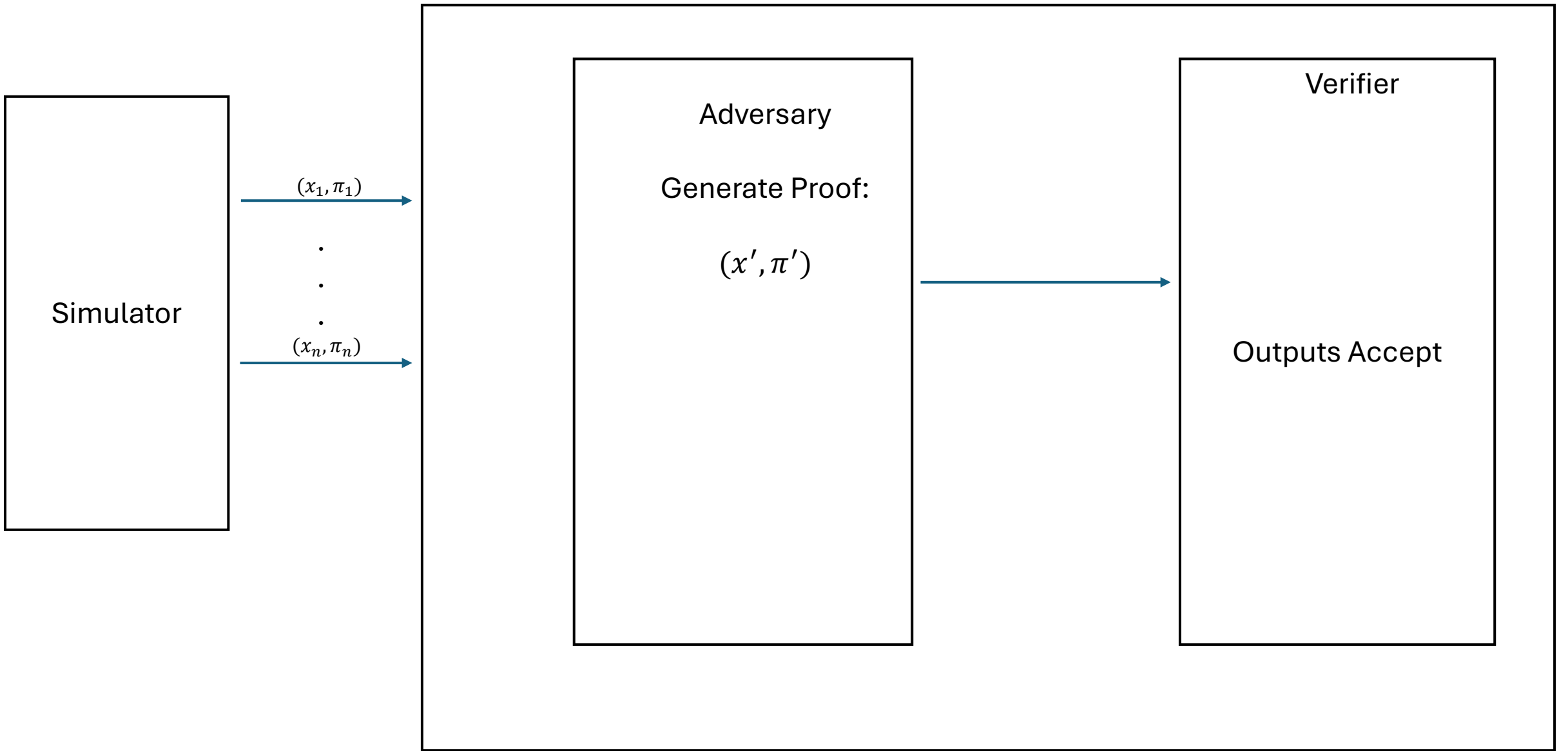
## Simulation Soundness: Reduction Argument

Assume towards contradiction:

- ▶ There exists PPT  $B$  that outputs valid  $(x, \pi)$
- ▶ Extractor  $K$  fails with non-negligible probability

Construct adversary  $A$  for the *basic* extraction game:

- ▶ Internally runs  $B$
- ▶ Answers  $B$ 's proof queries using the real prover
- ▶ Forwards all oracle queries unchanged
- ▶ Outputs  $B$ 's final  $(x, \pi)$



Simulation Soundness game

Basic Extractor

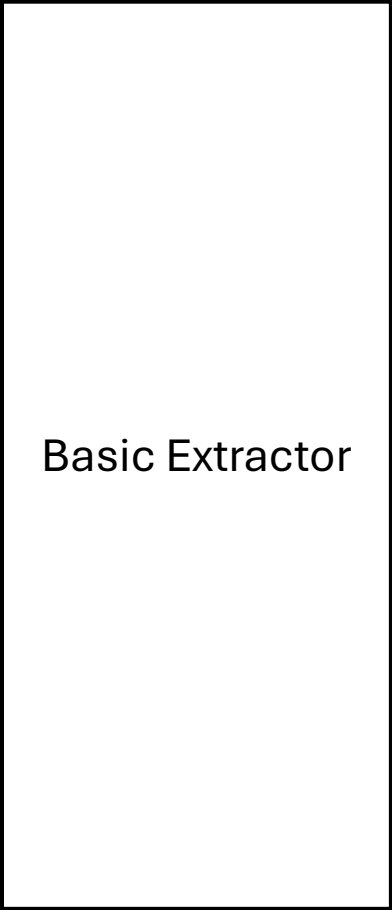
Ext adversary A

To answer proof queries  
of B.

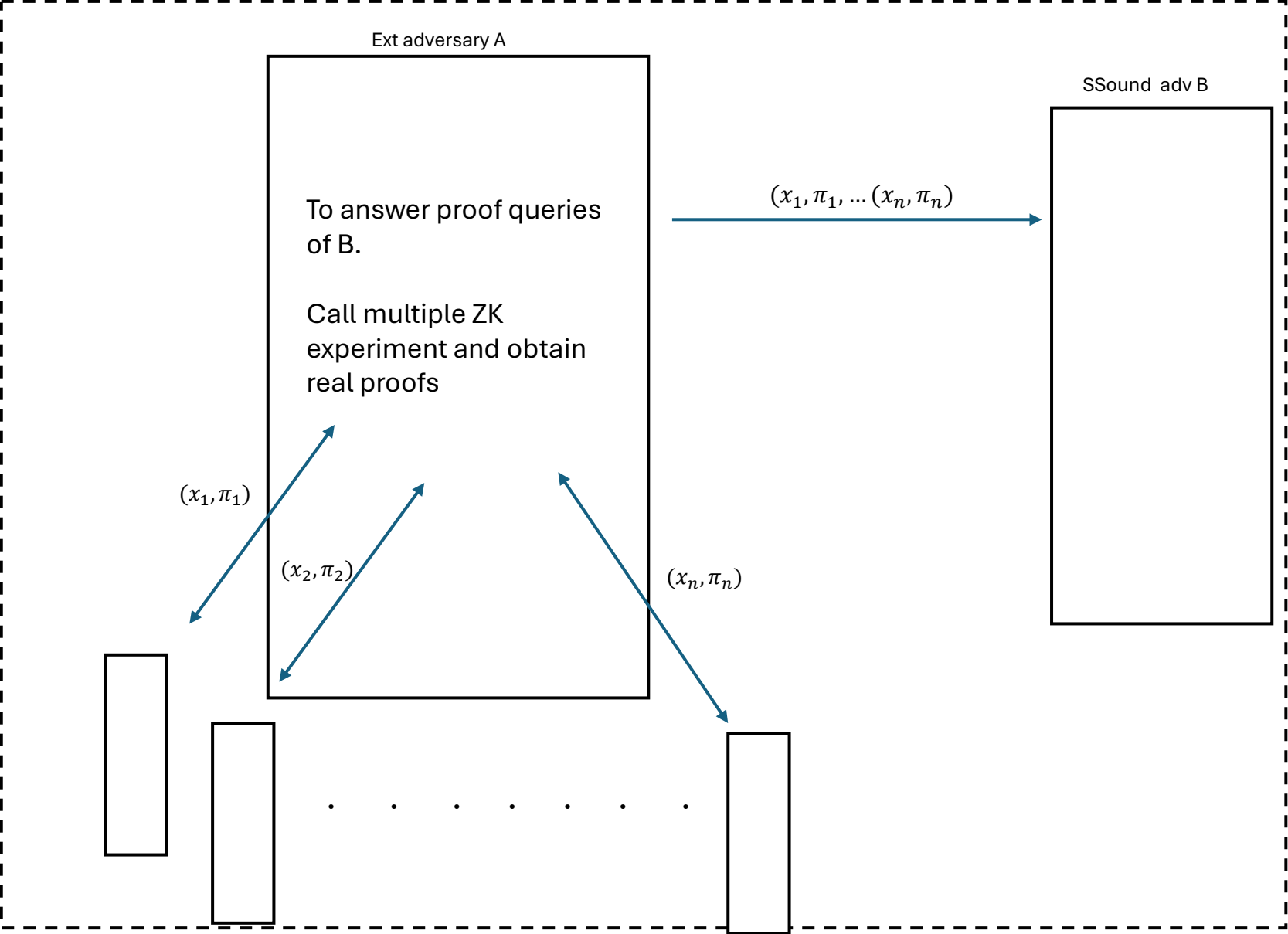
SSound adv B

Send the proofs

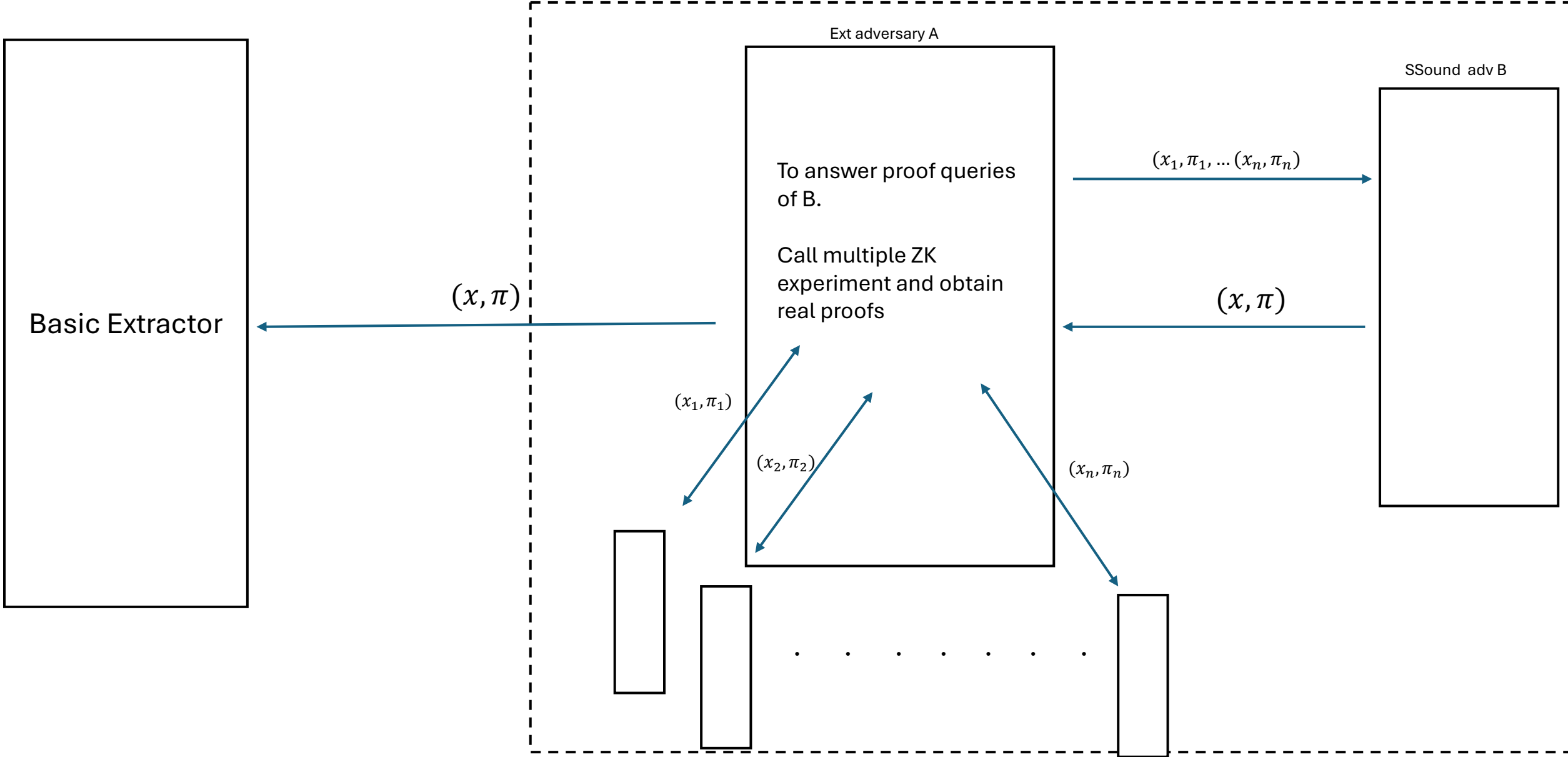




Basic Extractor







## Simulation Soundness: Why the Reduction Works

- ▶ By multiple zero-knowledge:
  - ▶ The view of  $B$  with simulated proofs
  - ▶ Is indistinguishable from the view with real proofs
- ▶ The extractor  $K$ :
  - ▶ Only depends on oracle queries for the final  $(x, \text{com})$
  - ▶ Queries related to earlier simulated proofs are irrelevant
- ▶ Hence extraction failure transfers to the basic setting

## Simulation Soundness: Conclusion

- ▶ Extraction failure in the basic setting is negligible
- ▶ Otherwise, one could distinguish real and simulated executions
- ▶ This contradicts multiple zero-knowledge

$\Rightarrow$  Simulation soundness holds

**Thank you**

## Step 1: Definition of the Online Extractor

- ▶ The extractor receives:
  - ▶ the statement  $x$
  - ▶ the proof  $\pi$
  - ▶ the list of hash queries made by the adversary
- ▶ It searches for:
  - ▶ two accepting queries
  - ▶ with same  $(x, \vec{com}, i)$
  - ▶ but different challenges
- ▶ If found: extract witness
- ▶ Otherwise: output failure

## Step 2: When Can the Extractor Fail?

- ▶ Extraction succeeds whenever two accepting challenges exist
- ▶ Failure happens only if:
  - ▶ the verifier accepts the proof
  - ▶ but no such pair of challenges exists
- ▶ The proof bounds the probability of this bad event

## Step 3: Fixing a Commitment Tuple

- ▶ **Assumption:** Adversary  $A$  never queries for two accepting transcripts in the same round.
- ▶ Fix one tuple  $(x, \vec{com})$
- ▶ Analyze acceptance probability for this fixed tuple

## Step 4: Why Challenges and Responses Are Unique

- ▶ If two different challenges existed:
  - ▶ extraction would already succeed
- ▶ Hence, each index contributes at most one valid value



## Assignment of Random Variables $s_i$

For each round  $i$ :

- ▶ **Queried by adversary:**

$s_i = \text{RO output previously returned to } A$

- ▶ **Never queried:**

$s_i = \text{fresh random } b\text{-bit string}$

- ▶ **Observation:** All  $s_i$  are independent.

## Step 5: Why Acceptance Is Unlikely

- ▶ The verifier accepts only if:

$$\sum_{i=1}^r s_i \leq S$$

- ▶ Each  $s_i$  is random and large
- ▶ The sum of many random values is unlikely to be small
- ▶ This is the key soundness amplification step

## Step 6: Union Bound Over All Commitments

- ▶ There are polynomially many candidate commitments
- ▶ Apply a union bound over all of them
- ▶ The total failure probability remains negligible

## Final Takeaway

- ▶ Either:
  - ▶ the adversary creates two accepting challenges
  - ▶ extraction succeeds
- ▶ Or:
  - ▶ all hash outputs behave randomly
  - ▶ acceptance probability is negligible
- ▶ Hence, every accepting proof implies knowledge